SPOT Manual

(Version 1.1.4)

Chapter 1 - About SPOT

This chapter will help with the decision to adopt SPOT or not, as a helpful tool for your orchestration tasks.

1.1 What is SPOT

SPOT stands for Simple PowerShell Orchestration Tool and, as the name suggests, it is meant to be simple to use while at the same time providing an extended set of features, that can be learned and used progressively. The tool is targeted for people that have a fair understanding of PowerShell functions/scripts (referred to as step functions in this manual) and use them in their orchestrations.

What are the key problems that SPOT solves?

PowerShell provides a complex environment capable of many features used in orchestration, like: remote execution (connect and execute code over WinRM, WMI or SMB), parallel execution (by using Runspaces, RunspacePool), cross-platform execution (connect and execute code over SSH/Telnet), execution sequences (execute existing PowerShell and cmd scripts), secure handling of secrets (SecureStrings, PSCredentials), error handling, etc. The problem is that all these functionalities require the orchestration scripts to be adapted also to these specific tasks (remote execution, secret handling, etc.), instead of focusing entirely on the main task for which they are created (e.g. configuring a DHCP server).

In this context, SPOT provides a homogenous step function execution system that allow the user to focus on developing the orchestration code that he needs and not, for example, on the options to execute code remotely, to manage centralized function output, to transfer variables between step functions, to handle secrets in a secured manner, both locally and remote, to handle retries, etc.

SPOT helps with organizing sets of already developed functions and scripts into SPOT projects, from where they can be launched and executed locally or remotely in controlled sequences, making it suitable also for CICD pipelines.

Besides PowerShell, the orchestration scripts can also consist of cmd/bat, SSH and Telnet scripts, making SPOT also suitable for the orchestration of other types of operating systems (Linux and similar) or appliances (various network devices with SSH or Telnet capability).

The orchestration sequences, called Runbooks, are defined in yaml files, making them portable and easy to understand/edit.

The SPOT execution is non-persistent and on-demand, so there is no service or database needed for SPOT.

1.2 Functionality Overview

The mere possibility to execute several PowerShell scripts in a certain sequence is not something special as this can be easily achieved by creating a calling script with the desired sequence of called scripts inside. Maybe mixing local and remote scripts in such a calling script could be a little bit more challenging, but not by much. The core functionalities that SPOT provides go a little bit further than the calling script model:

- Each step (called a Runbook Step), from a sequence (called a Runbook) is executed only if the previous step was evaluated as successful. That is unless this behavior is overriden by specific config options. However, the principle of progressing through a sequence only if each steps is successfully executed remains very important.
    - How are the steps evaluated? If during the execution there is no terminating error detected, the step is considered successfully executed.
    - There is also the possibility to configure Conditions on a Runbook Step, which are evaluated at their execution time. If these Conditions are not validated, the respective Step is skipped and the execution progress goes to the next step in the Runbook.
    - In case the step function contains “exit” commands to signal the success or the failure of the execution, the exit code will be honored: the exit command will be intercepted and any non-zero exit code will trigger a terminating error.
    - There is a step configuration option that can override the normal stop of the orchestration if a step is not successfully executed, which is called ContinueOnError.
- A Runbook can be configured as a Runbook Step in another (parent) Runbook, thus allowing for Runbook nesting. When this happens, the Runbook properties defined in the parent RunbookStep have priority and overwrite the ones defined in the standalone Runbook (in the Runbook yaml file). There is no imposed or tested limit to the number of nesting levels. By default, all Runbook executions, nested or not, are done locally on the SPOT Computer. There is a set of remote parameters that can be configured on a nested Runbook, that allow for remote execution of that Runbook (and any child Runbooks included). In this case, the remote target is minimally configured with a temporary SPOT environment, which is automatically removed at the end of the execution. This feature allows the orchestration to be decentralized if needed, in order to cover targets that are not reachable directly from the SPOT Computer, or in order to amplify the parallel SPOT execution capacity.
- Multiple steps can be executed in parallel if they are properly configured for this. There are two types of parallel steps executions possible in SPOT:
    - For different step functions to be executed in parallel on various targets, the respective RunbookSteps/Runbooks need to have the same Seq number inside a parent Runbook.
    - For the same step function to be executed with the same parameters on multiple targets, the target parameter of the respective step must be configured with a collection of the target addresses (hostnames or IP Addresses). In this case, since a single step

object is used inside the Runbook for a parallel execution, the result of the execution must be reflected in this object based on the overall success of the parallel jobs inside. For these cases, the SPOT project preference _AnyFailFail was defined and it is described below.

    - To allow for parallelism, all steps and all child Runbooks are executed in their own separate PowerShell Runspaces.

- For the PowerShell steps, the step function (originating in custom scripts and functions) can be defined and then configured inside the steps. During the Main Runbook loading phase, all script files are loaded from the "_Scripts" project folder and then converted to functions. So, in the end, only PowerShell functions are actually manipulated in the PowerShell Runbook Steps.
- The full execution output from all Runbook Steps is saved automatically in a file and folder structure resembling the Runbook and Runbook Steps names, inside the "__SPOT_Artefacts" folder of the current project. The output log of each step is made available in this folder only after the execution has been completed. If logging to additional files in real time is desired, custom log functions can be defined as part of the project. There is also a built-in logging function used inside SPOT called Write-SPOTLog, which logs the internal SPOT activity to a predefined file path: C:\Windows\temp\Orchestration.log.
- Before executing a Runbook, it must be loaded. During loading, all Runbook Steps have their parameters validated (to make sure that the mandatory step parameters and mandatory parameters for the step function are defined). Besides validation, any parameters containing references to the SecretVault or OrchVars are substituted with the intended value, while for any parameters containing references to local files/folders (RFI and RFO) their file system path is tested.
- The secrets used in SPOT are managed within vaults (with the help of Microsoft Secret Management PowerShell modules) and secured variables (Secured Strings or PS Credentials).

1.3 System Requirements

PowerShell 5.1

.NET Framework 4.7.1 or higher

Operating System

- In theory: any OS with PowerShell 5.1
- Tested on:
    - Windows Server 2016
    - Windows Server 2019
    - Windows Server 2022
    - Windows 10
    - Windows 11

Chapter 2 - SPOT Installation

The SPOT tool is packaged as a PowerShell Module and has several other modules and tools as dependencies.

There are two main installation methods available, to handle the main module and the dependencies: online repository method and scripted installer. The first method can only handle online installations while the second method can handle both. It will be a matter of need as well as personal preference when choosing an installation method out of the two.

Important: Out of the SPOT dependencies, PsExec is a special case as it has an End-User License Agreement that needs to be accepted. This EULA is triggered automatically during the offline installation of STOP or, for the online installations, during the extending of the SPOT Capability (e.g. with the command Extend-SPOTCapability). After an initial acceptance of the PsExec EULA, the tool will be used completely unattended even on remote computers, if the orchestration is configured to use PsExec remotely. Technically, this is achieved by using the built-in PsExec command line argument "-accepteula".

2.1 Repository method

For the method using the PowerShell Gallery repository, the online Microsoft documentation is available here: https://learn.microsoft.com/en-us/powershell/gallery/getting-started?view=powershellget-3.x

As mentioned in the documentation above, for installing SPOT, the command would be:

Install-Module -Name SPOT

The SPOT module dependencies would be installed automatically and the remaining SPOT tool dependencies (SshNet and PsExec) would have to be installed by using the following SPOT command:

Extend-SPOTCapability -Online

The SPOT Capability can be extended also offline after an online initial SPOT installation. For this to be done, the two optional tools used to extend the SPOT Capability must be downloaded manually from the following URLs and saved for later use as zip files:

https://www.powershellgallery.com/api/v2/package/Posh-SSH/3.2.7 - for SshNet

https://download.sysinternals.com/files/PSTools.zip - for PsExec

After the above optional tools are available, the following command can be used to extend the SPOT Capability offline (local archive paths to be adapted):

Extend-SPOTCapability -PsToolsZipPath "C:\temp\PSTools.zip" -PoshSSHZipPath "C:\temp\posh-ssh.3.2.7.zip"

Depending on the PsExec EULA agreement and any potential errors during the extension, the SPOT Capability can remain “Core” (no optional tools is available), “SshNet” (only the SshNet libraries are available) or “Extended” (both SshNet and PsExec tools are available).

Technically, after extensions, the optional tool files are stored in the “Tools” subfolder from the SPOT Powershell module directory.

Because the SPOT Module installation folder for online installs is separate for each version (to support side-by-side versions), any new installed/upgraded SPOT version requires a new SPOT Capability extension, if desired.

2.2 Script Installer method

For the installer method, the installer script “SPOT-Installer.ps1” provided with SPOT may be used. There are 2 modes of installation with this method, depending on the conditions:

- **Install From Internet**, which is also an online installation method, but with the difference that all SPOT dependencies (modules and tools) are installed. The command to use would be:
  - *.\SPOT-Installer.ps1 -InstallFromInternet*
- **Install Offline**, which is done in 2 steps as follows:
  - Download all SPOT related packages to create a big, offline, SPOT thick package.
    - *.\SPOT-Installer.ps1 -CreateSPOTPackage -SPOTPackagePath C:\temp\SPOT.zip*
  - Install offline from the thick package provisioned in the first step.
    - *.\SPOT-Installer.ps1 -InstallFromSPOTPackage -SPOTPackagePath C:\temp\SPOT.zip*

NOTICE: There are some optional extra parameters available from the SPOT-Installer.ps1 script, for special cases:

- “SPOTVersion” which is for cases when the desired version to install is not the latest. It is usable only alongside “InstallFromInternet” and “CreateSPOTPackage” parameters. If not specified, the latest version is implied.
- “Extended” which offers the possibility to install SPOT offline without attempting to extend the SPOT Capability. If not specified, the Extended installation mode will be attempted.

Because the SPOT Module installation folder for offline installs is separate for each version (to support side-by-side versions), any new installed/upgraded SPOT version requires a new SPOT Capability extension, if desired.

2.3. SPOT initialization

After the SPOT installation, in order to use the tool properly and create projects, SPOT must also be initialized. This initialization is performed on two aspects:

- **SPOT status initialization**, which is done at the OS level and makes sure that the main OS requirements are configured to support SPOT (local settings for PSRemoting and Firewall). The master password supplied now is placed in encrypted form inside an environment variable of the current user and its purpose is to be used as a Secret Vault password. The command to use for this case is:
    - *Initialize-SPOT -MasterPassword MyPassword*

  For production environments, where the SPOT application can be used as an orchestration tool (both in command line mode and in GUI mode), the prestaging of the password could be considered a vulnerability and this is why the master password could be provided manually on every command that may require access to the Secret Vault. In this situation SPOT must be initialized with the following command:

    - *Initialize-SPOT -MasterPassword MyPassword -RegisterMasterPassword $false*

  Of course, the password used above is just an example, but what is important is that it must be typed in in clear text.

  The MasterPassword prestaging can also be performed separately at any time, before or after a SPOT initialization, in order to prestage later the MasterPassword or to change the already prestaged MasterPassword. This can be done with the command:

    - *Register-SPOTMasterPassword -Password MyPassword*

  The Initialize-SPOT command is also performing a SPOT Secret Store initialization, in case the SPOT Secret Store is not already initialized. But the SPOT Secret Store initialization can also be done separately, at any time, as explained next.

- **SPOT Secret Store initialization**, which is done at the Secret Store PowerShell module level, for the current user on the current computer, to have it configured to support SPOT. The command to use for this initialization is:
    - *Initialize-SPOTSecretStore*

  If the master password is not prestaged on the system, the following command should be used instead:

    - *Initialize-SPOTSecretStore -MasterPassword MyPassword*

  NOTICE: This SPOT Secret Store initialization is removing any previous vaults registered for the current user on the current computer.

2.4. SPOT status handling

Before or after the SPOT initialization procedures, the overall details about SPOT can be visualized with the following command:

Show-SPOTDetails

2025-12-26 04:45:56 :: ===== Starting function Show-SPOTDetails =====

2025-12-26 04:45:56 :: SPOT version : 1.1.1

2025-12-26 04:45:56 :: SPOT Status : NotInitialized

2025-12-26 04:45:56 :: SPOT Capability : Extended

2025-12-26 04:45:57 :: SPOT Secret Store : NotInitialized

2025-12-26 04:45:57 :: PsExec version : 2.43

2025-12-26 04:45:57 :: SshNet version : 2024.0.0.0

2025-12-26 04:45:57 :: ===== Finished function Show-SPOTDetails =====

The output from this command provides an overview about the status of various SPOT components. They can also be interrogated separately using the following commands:

- *Get-SPOTStatus*, which returns the status of the SPOT installation. The possible return values are: NotInitialized, Intialized, Incomplete or MissingDependencies.
- *Show-SPOTCapability*, which returns the Capability status of SPOT. The possible return values are: Core, SshNet or Extended.
- *Get-SPOTSecretStoreStatus*, which returns the status of the SPOT SecretStore. Since this is related to the Secret Vault, the MasterPassword parameter is necessary if the master password was not prestaged inside the OS. The possible return values are: NotInitialized or Initialized.

Chapter 3 - Manage SPOT settings

The OrchVars name stands for Orchestration Variables and they are a collection of variables that are defined in a project based file called OrchVars.yaml and initialized during the loading of a project into a synchronized hashtable. This OrchVars hashtable is made available to all scripts and functions executed as part of the SPOT orchestration. Values from this hashtable can be used both as parameters in the runbooks as well as inside the scripts and functions.

Some of the OrchVars are used as project configuration options and if they are not defined in the OrchVars.yaml file, default values are assigned to them in the OrchVars hashtable automatically during project initialization. The names of these special OrchVars are always starting with _ (underscore). This is why it is advisable that any custom OrchVars defined for a project should not start as well with _ (underscore). There would be no conflict or limitation, but it would just be harder to overwrite by accident a project configuration option and also be easier to differentiate in the event of troubleshooting.

Below is a complete list of **OrchVars configuration options/variables**:

_StopFlag: is used only during GUI execution and it signals to the runbook executing function that a STOP event has been triggered. After this flag is set to True, during a runbook execution, on the next occasion where a step execution is finished, the runbook execution is halted in a resumable position.

- Should not be used inside the OrchVars.yaml file.
- Default value: $false.

_SPOTPath: contains the local path to the SPOT module folder, on the SPOT Computer. In remote SPOT environments, this variable is null because there is no expected SPOT folder there.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_ProjectPath: is populated during SPOT project loading with a value taken from a parameter and contains the local path to the current SPOT project folder. Based on this parameter, a specific project OrchVars.yaml file is processed next. In remote SPOT environments, the entire project folder is made available into a temporary folder and this variable contains this temporary folder path.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_RetryCount: can be defined with an integer value in the OrchVars.yaml file. It configures the number of retries that are attempted for every runbook step in case the step execution ends with an error. This option can also be configured directly on runbook steps, in which case this value overwrites the project configuration option for that step. When configured on a runbook step, the option is called *RetryCount* (without underscore).

- Can be used inside the OrchVars.yaml file.
- Default value: 0.

_RetryDelay: can be defined with an integer value in the OrchVars.yaml file. It configures the number of seconds that are waited between any eventual execution retries. This option can also be configured directly on runbook steps, in which case this value overwrites the project configuration option for that step. When configured on a runbook step, the option is called *RetryDelay* (without underscore).

- Can be used inside the OrchVars.yaml file.
- Default value: 30.

_StepTimeout: can be defined with an integer value in the OrchVars.yaml file. Basically, this sets an overall timeout in seconds for the remote steps that are asynchronous (PowerShellCommandRemoteSJ, PowerShellCommandRemoteWMI, PowerShellCommandRemoteOWMI and PowerShellCommandRemotePsExec).

- Can be used inside the OrchVars.yaml file.
- Default value: 3600.

_ContinueOnError: can be defined with a Boolean value in the OrchVars.yaml file. It configures the behavior of the runbook execution when a runbook step execution ends in error. This option can also be configured directly on runbook steps, in which case this value overwrites the project configuration option for that step. When configured on a runbook step, the option is called *ContinueOnError* (without underscore).

- Can be used inside the OrchVars.yaml file.
- Default value: $false.

_AnyFailFail: can be defined with a Boolean value in the OrchVars.yaml file. It configures the behavior of the runbook execution for the specific case of implied parallel step executions. The implied parallel step executions occur only for remote execution steps, where the RemoteComputer parameter is populated with an array of values or with a comma separated string list. In these cases, the parallel executions could lead to mixed results, where some of them are successful and some of them end with an error. However, these parallel executions are represented in their parent runbook with a single step and a decision has to be taken after all parallel executions finished, with regards to the entire step execution result. If the _AnyFailFail configuration option has a value of False, then the runbook step will be considered successful unless all parallel implied executions end in error (Any Fail will NOT lead to overall Fail). Otherwise, if the _AnyFailFail configuration option has a value of True, then the runbook step will be considered successful only if all parallel implied executions are successful (Any Fail will lead to overall Fail).

- Can be used inside the OrchVars.yaml file.
- Default value: $false.

_VaultName: can be defined with a string value in the OrchVars.yaml file. This value sets the prefix used for the names of the secrets inside the default Vault, so that multiple projects can share the same vault and also have the same secret names with different values inside.

- Can be used inside the OrchVars.yaml file.
- Default value: the name of the current SPOT project.

_Debug: it configures the SPOT built-in logging function Write-SPOTLog to execute the writes marked with the DBG flag, basically extending the logging from the default minimal level to a very detailed level, suitable for debug situations.

- Can be used inside the OrchVars.yaml file.
- Default value: $false.

_PsExecPath: contains the local path to the PsExec.exe tool, on the SPOT Computer. In remote SPOT environments, this tool is made available into a temporary folder and this variable contains this temporary folder path.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_SshNetPath: contains the local path to the Renci.SshNet.dll tool, on the SPOT Computer. In remote SPOT environments, this tool is made available into a temporary folder and this variable contains this temporary folder path.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_SPOTCapability: contains the value of the SPOT Capability property. It can be “Core” (no extended SPOT tools available), “SshNet” (SshNet tool available) or “Extended” (both SshNet and PsExec tools available). This value is detected during project loading and is used during Runbook loading to make sure there are no steps included that depend on missing extended SPOT tools.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_SPOTClassDef: contains the SPOT Classes definition code (for the Runbook and RunbookStep objects), encoded as a base64 string. This is needed to make possible the use of the SPOT objects in remote SPOT environments.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_SPOTHostFunctions: contains the custom function definition representing the main Host related PowerShell commands and used to overwrite them for compatibility with the runspace environment (the $host object is not available inside runspaces). This content is also encoded as a base64 string.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_RFOMap: contains the mappings between referenced folders for remote steps and their unique IDs, to distinguish between them while making them available remotely to the step function.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

_SPOTRsPoolMax: can be defined with an integer value in the OrchVars.yaml file. It sets the maximum number of parallel Runspaces in the RunspacePool used for remote step executions. This is a performance related setting so it depends on the resources available on the SPOT computer, or of the remote SPOT computers in case of remote runbook executions. A rule of thumb here is 5 x the number of available Cores, when going beyond the default value of 20, which should be enough for most cases.

- Can be used inside the OrchVars.yaml file.
- Default value: 20.

_ProjectFunctions, *_StepFunctions* *and* *_RunbookFunctions*: are lists of function names determined during SPOT project initialization. These lists assign the SPOT internal functions and SPOT Project user functions to separate uses.

- Should not be used inside the OrchVars.yaml file.
- Default value: detected automatically.

Chapter 4 - Using SPOT

4.1 Create Projects

To create a new SPOT Project, the command “Initialize-SPOTProject” can be used. The simplest SPOT project will be initialized in the desired Project path, ready to receive new content.

Initialize-SPOTProject -ProjectPath “C:\TemplateProject”

The only files created are “OrchVars.yaml” for SPOT project settings and orchestration variables and “SInputs.yaml” for secret definitions. They are both located inside the “__SPOT_Config” project folder and empty by default.

If the SPOT master password is not prestaged, the project initialization command must specify it.

Initialize-SPOTProject -ProjectPath “C:\TemplateProject” -MasterPassword MyPassword

After a new SPOT project is initialized, it can be modified with new OrchVars, Secrets, Scripts, Functions, Runbooks, as well as other custom files and tools, as they are needed.

NOTICE: If a SPOT project or any other content is already present in the ProjectPath provided to the Initialize-SPOTProject command, it will be not be automatically deleted and the command will return an error.

4.2 Populate OrchVars and Secrets

OrchVars

The purpose of OrchVars is to provide a common place for all sorts of configuration parameters needed for an orchestration (which may contain one or more Runbooks).

The SPOT project configuration file “OrchVars.yaml” is located inside the “__SPOT_Config” project folder and can contain both project settings (from the detailed list from chapter 3) as well as custom orchestration related configuration values.

As this is initially a yaml file that gets loaded into a Powershell hashtable, it can provide structure to the parameters, with sections and multiple levels.

The SPOT project settings, if any, must be placed in the first level of the yaml file. If a setting is not specified, its default value will be used automatically.

The orchestration configuration values can be defined on multiple levels inside the yaml file and will be accessible in the step parameters and inside the step functions by their full reference location, similar to accessing nested hashtable variable values.

- For step parameters, the syntax is like this: $OV:Site, $OV:TestVarCollection.TestVar (more details about step parameter referencing will be provided later in this document)
- For inside step functions, the syntax is like this: $OrchVars.Site, $OrchVars.TestVarCollection.TestVar

In the file form, "OrchVars.yaml" can only contain text/string values for the variables, but with the help of referencing, these values can extend to credentials and secured strings from the secret vault. So the referencing of Secrets can be made inside the "OrchVars.yaml" file itself with the same syntax (e.g. $SV:SrvCred, where SrvCred is the store name of the intended secret). The limitation is that this referencing can only be made 3 levels deep inside the "OrchVars.yaml" file.

Below is a sample with 3 levels, that also includes some settings and some $SV references.

```yaml
TestVariable: TestValue
TestVariable2: TestValue2
#_ClassicMode: true
TestCredential: $SV:SrvCred2
_Debug: true
ADConfig:
    DomainName: contoso.com
    DCSafeModePass: $SV:DCSafeModePass
SCOMConfig:
    SCOMGroupName: Nexus
    SNMPV2Comm1:
        Name: CommunityString1
        String: $SV:CS1
    SNMPV2Comm2:
        Name: CommunityString2
        String: $SV:CS2
```

Secrets

All remote execution step types require *administrative* credentials for the target remote systems. But this is not the only place to use secrets. They can be used also for defining alternative RunAs accounts, for step function parameters or for defining OrchVars entries. The management of all secrets in a SPOT project is centralized. The SPOT project configuration file "SInputs.yaml" is located inside the "__SPOT_Config" project folder as well and can contain two types of secrets: string secrets and credentials.

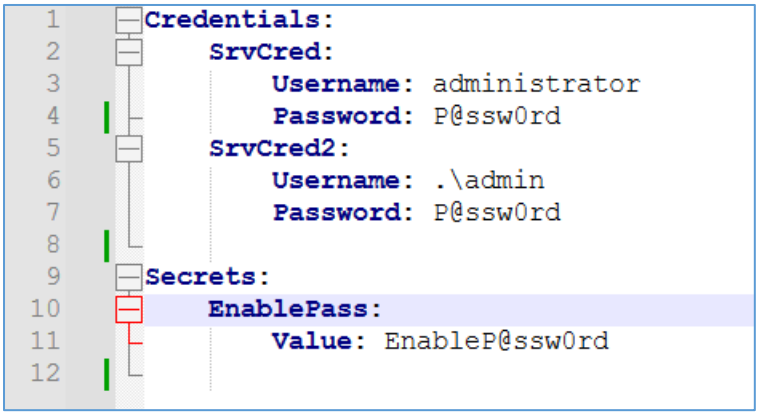
The SPOT application includes a secret store with a default Secret Vault configured to be shared amongst the SPOT projects loaded on the current computer and under the current user. It is important to note this detail about the current user, because this means that the Secret Vault needs to be initialized for every user that is used on a computer for SPOT runbook executions and that the Secret Vault will be different between users. The Secret Vault initialization can be triggered at any time by executing the SPOT internal function "Initialize-SPOTSecretStore" for each applicable user on a SPOT computer. If the SPOT master password is not prestaged the SPOT Secret Store initialization command must specify it.

During Secret Vault initialization, it gets (re)created from scratch using the prestaged (under an environment variable) or provided (as a parameter) master password, then it gets configured with the default password timeout of 180 seconds. No secrets are injected into the Secret Vault during initialization.

The population of the Secret Vault with secrets happens in two cases:

1) Automatically during every execution of a SPOT Runbook, either with the command line method or with the GUI method (during project loading), as long as the default secrets file "SInputs.yaml" is present in the "__SPOT_Config" project folder.
2) On demand, when the SPOT function "Import-SPOTProjectSecrets" is executed. Executing this function on demand allows the option to specify a different path and filename for the secrets file (the parameter "SecretsFilePath"). If using this option, the default secrets file must be deleted from the default path in order to avoid erasing the populated secrets automatically from the project Secret Vault, due to the automatic nature of option 1 above.

The secrets input file, either the default or a custom one, is structured into two sections, one for credentials and one for string secrets (e.g. enable passwords). See sample below.



It is important not to have duplicate secret names and to keep all the time the same structure under each element (Username and Password for credentials and Value for secrets).

Before every execution of a SPOT Runbook, either with the command line method or with the GUI method (during project loading), the Secret Store is unlocked automatically and all secrets (secured strings and credentials) from the project Secret Vault are loaded into the $SVars internal SPOT variable.

The Secrets populated in the SPOT Secret Vault can be managed using the following commands:

- *Import-SPOTProjectSecrets*, which provides a method to import project secrets either from the default project secrets file (SInputs.yaml) from the "__SPOT_Config" project folder or from a custom location. Reminder, the import of secrets from the default project secrets file happens automatically during each SPOT execution if the secrets file is present in its project folder.
- *Show-SPOTProjectSecretsInfo*, which provides a method to visualize secrets medatada for secrets associated with specific projects, vaults or all.
- *Remove-SPOTProjectSecrets*, which provides a method to cleanup the secrets associated to specific projects or project vaults.

4.3 Populate functions and scripts

The scripts and functions defined in a SPOT project are the most basic building blocks of the orchestration. That is because the SPOT tool is built around the idea of controlling the sequenced execution of PowerShell scripts or functions.

There is also a small collection of step functions provided by SPOT, to provide the basic orchestration elements for interacting with Linux based operating systems or network appliances over SSH and Telnet protocols.

All step functions from a SPOT project, including the built-in ones, can be listed (and then inspected) with the following command:

Get-SPOTProjectFunctionList -ProjectPath "C:\TemplateProject"

In a SPOT project, the PowerShell scripts must be placed under the "_Scripts" folder and the PowerShell functions (defined inside PowerShell function files) must be placed under the "_HelperFunctions" folder. Both these folders can be organized with subfolders. During the loading of a SPOT project, all PowerShell script files detected under the "_Scripts" folder are converted into functions and added to an array of SPOT project functions, together with the PowerShell functions loaded from the function files. They must be kept separate from each-other because of the different way they can be loaded or converted inside SPOT.

Since all scripts and functions converge (through loading or converting) into a set of SPOT project functions, it is important to distinguish between two main types of SPOT functions:

- Step Functions which are functions created to be executed inside Runbook Steps. There is no connection between this classification of functions and the files from where they are loaded. Step Functions can be loaded from scripts as well as from helper function files.
- Helper Functions which are functions created to improve reusability of code inside the Step Functions. As the name suggests, these are best suited to be placed inside the function files from the "_HelperFunctions" folder, from where they can be used both in SPOT or reused in other environments. But this is just a recommendation and not a mandatory condition.

The step scripts and functions are the basic elements of the orchestration that the user develops and integrates into SPOT projects. There is that small collection of step functions provided by SPOT, but that is an exception and more of a showcase of what can be integrated inside SPOT.

When developing the step scripts and functions, there are a few guidelines that should be observed:

- Avoid using variable names starting with "$_spot_" because all internal SPOT variables exposed to the same execution environment as the step functions use this naming convention.
- Avoid using the "$Using" variable as this may break the scripts conversion of the functions loading. It should not be needed anyway, given that SPOT already offers several remote execution mechanisms including any needed parameters.
- Be aware that inside the step scripts and functions the $OrchVars SPOT variable can be called, for both local and remote executed steps. Incorporating $OrchVars calls in these scripts and

functions can make them SPOT dependent, but to avoid this, you can simply define internal variables at the beginning and assign them values from the $OrchVars only if $OrchVars are available.
- The secret variables ($SVars) and published variables ($PublishedData) are not available consistently inside the step scripts and function and as such, they should not be used.

All SPOT script files (internal and built-in step functions) have encoding set to UTF-8. It is recommended to keep all script/function files encoding aligned as there are some functions (e.g. Export-SPOTDevHelperFile) that collect all project scripts/functions and some SPOT internal functions into a single DevHelper file and mixing encoding into a single file is not advisable as some characters can be interpreted different (the most frequent to be affected are " - " and " ' ").

During a SPOT Project loading, the encoding of all script and helperfunction files is checked and a warning is logged in case some of these files are not UTF-8 encoded.

4.4 Using SPOT built-in helper functions

When creating scripts and functions to be used as step functions inside SPOT, there are a few Helper functions defined inside SPOT but made available during step function execution. These are primarily used inside the SPOT built-in step functions (documented below), but can be used also directly in user-made step scripts and functions.

- **Ping-SPOTHostWMI**
  - Simple pinging function based on WMI
  - The only parameter is the target IP Address
- **Test-SPOTTCPPort**
  - Simple TCP Port tester, used primarily inside SPOT
  - The only parameters are: the target IP Address and the target Port
  - Returns a hashtable with the TCP Port test result as well as with a Ping test result
- **New-SPOTSFTPSession**
  - Wrapper function to create an SshNet SFTP session object
  - Should be primarily used only in locally executed step functions
  - The usual self-explanatory parameters are present:
    - TargetIP
    - Port
    - Credential
  - Two additional, custom attributes are also available:
    - SshNetPath which should be left for SPOT to manage (if step function is executed remotely, this parameter must be exposed in the step function and populated with "$RFO:SSHNET" so that the SshNet files are made available remotely during execution)
    - TrustedHostsFilePath which is a reference to a SPOT Trusted Hosts csv file. This file can be populated based on SSH Key values manual lookup in the SPOT logs during a safe connection to the target host or based on actual SSH Key files, with

the help of the public SPOT function “Add-SPOTSSHTrustedHostKey”. If this parameter is not specified, the SSH Key validation is not enabled.

- **New-SPOTSSHSession**
  - Wrapper function to create an SshNet SSH session object
  - All the other details from “New-SPOTSFTPSession” are also applicable here.
- **New-SPOTTelnetSession**
  - The only parameters are: the target IP Address and the target Port
  - Custom made function to create a simple Telnet session object, with a few custom methods exposed:
    - WriteLine – for sending commands
    - ReadOutput – for getting command output
    - RemoveSession – for disconnecting and disposing the Telnet TCP connections

4.5 Create Runbooks

The next level of building blocks inside SPOT, after the step scripts and functions, are the Runbooks. They are also yaml files and must be placed inside the “__SPOT_Runbooks” folder, where they can be organized in subfolders.

Each Runbook file controls the execution of a series of steps in a sequence, based on the “Seq” step attribute. This “Seq” number has an influence only in the current Runbook and not in other parent or child Runbooks. If multiple steps have the same “Seq” number, all these steps will be executed in parallel (after being started in a quick succession, each in its own runspace).

The Runbook file may contain a RunbookParameters section where the user can define a set of parameters which will apply only inside that Runbook. They can be referenced in the runbook steps just like referencing other variable types. This feature can make a runbook behave like a function in the context of the orchestration, allowing the re-use of a runbook with different parameters for different purposes or different environments. The re-use of a runbook can be done even in the same parent runbook, by specifying the RunbookName property in the step properties of the child runbook and avoid in this way a duplicate step name in the parent runbook.

4.5.1 Step types

The steps can be configured to be any of the step types defined in SPOT (in the file “\SPOT\Config\StepTypeDefinitions.yaml”, where also all possible step parameters are defined):

- **PowershellCommandLocal**
  - This step type is used for the local execution of PowerShell step scripts or functions. All local step executions are performed under the same user context as the one used for the SPOT execution.

- This is a synchronous step type because it is local, performed inside a separate runspace.
- The $OrchVars and $SVars are made available inside the step functions during execution so these variable names must not be used inside the step functions to initialize or change these variable values.
- The PublishedData variables are made available inside the step functions during execution in this step type as $_spot_PublishedData so this hashtable can be used with care to access Published Variables directly (e.g. for collecting several published SSH_Key objects and inserting them automatically in a SSH Trusted Hosts file).

- **PowershellCommandRemote**
  - This step type is used for the remote execution of PowerShell step scripts or functions over an admin PSSession to the target computer.
  - The main limitation in this case is the fact that the remote execution via a PSSession is performed in a limited environment (e.g. the access to the Windows Credential Manager is missing). However, the second hop commands are working, as this is allowed in admin PSSessions (https://learn.microsoft.com/en-us/archive/blogs/sergey_babkins_blog/another-solution-to-multi-hop-powershell-remoting).
  - There is a small performance penalty here in order to use the Admin PSSession because the remote WinRM service needs a restart and a 2 second wait time is used for this.
  - This is a synchronous step type based on the WinRM service. CAUTION: If the WinRM service is restarted during the step execution, the execution state gets broken.
  - The $OrchVars are made available inside the step functions during execution so these variable names must not be used inside the step functions to initialize or change these variable values.

- **PowershellCommandRemoteSJ**
  - This step type is used for the remote execution of PowerShell step scripts or functions over a Scheduled Job (initiated through a normal PSSession).
  - The user context for this step type is configurable, meaning that a user account can be used for connecting to the remote connection while another user account (with local admin rights) can be used for the execution on the remote computer. Execution as Local System is also possible.
  - The main benefit in this case, compared to the PowershellCommandRemote, is that the execution environment is not limited, even if the used PSSession is not administrative.
  - This is an asynchronous step type based on the WinRM service. CAUTION: If the WinRM service is restarted during the step execution, the execution state gets broken.
  - The $OrchVars are made available inside the step functions during execution so these variable names must not be used inside the step functions to initialize or change these variable values.

- **PowershellCommandRemoteWMI**
    - This step type is used for the remote execution of PowerShell step scripts or functions over the WMI/SMB protocols (WMI for remote execution and SMB for data transfer).
    - The main benefit for this step type is that the WMI and SMB ports are usually opened by default in the Windows firewall, providing a remote execution possibility without a pre-configuration of the firewall.
    - The main known limitation is that there is no access to the Windows Credential Manager.
    - This is an asynchronous step type, so the StepTimeout setting applies.
    - The $OrchVars are made available inside the step functions during execution so these variable names must not be used inside the step functions to initialize or change these variable values.
- **PowershellCommandRemotePsExec**
    - This step type is used for the remote execution of PowerShell step scripts or functions over the SMB protocol.
    - The user context for this step type is configurable, meaning that the execution can be made as the Local System and not as other configurable user account.
    - The main benefit for this step type is that it provides a remote execution possibility when even both the WinRM and WMI ports are disabled on the remote computer. SMB is one of the core protocols used in Windows and the firewalls are usually allowing it.
    - Another benefit is that with this method, the remote execution has access to the Windows Credential Manager.
    - The main downside for this step type is the fact that is relies on the PsExec tool, which is restricted in some organizations due to some vulnerabilities (mainly from previous versions; e.g. logging of entire command lines, including passwords in clear, in the Windows EventLog).
    - This is an asynchronous step type, so the StepTimeout setting applies.
    - The $OrchVars are made available inside the step functions during execution so these variable names must not be used inside the step functions to initialize or change these variable values.
- **PowershellCommandRemoteOWMI**
    - This step type is used for the remote execution of PowerShell step scripts or functions over the WMI protocol only (for both the remote execution and for the data transfer).
    - The main benefit for this step type is that the WMI ports are usually opened by default in the Windows firewall, providing a remote execution possibility without a pre-configuration of the firewall.
    - The main known limitation is that there is no access to the Windows Credential Manager.
    - This is an asynchronous step type, so the StepTimeout setting applies.

- The $OrchVars are made available inside the step functions during execution so these variable names must not be used inside the step functions to initialize or change these variable values.

- **Runbook**
  - This step type is used for the execution of an entire Runbook, which in this case is called a child Runbook. The entire execution of the child Runbook is treated by the calling Runbook as a step, waiting for it to finish successfully or executing it in parallel with other steps configured with the same "Seq" number. This feature can be called runbook nesting.
  - This step type can be configured with remote parameters, to be executed on a remote computer, basically allowing a remote computer to become a temporary SPOT orchestrator. The method of remote execution in this case is specified as one of the remote function types above. This feature is called remote runbook nesting.
  - One of the differences between the local runbook nesting and the remote runbook nesting is that the progress (in both command line as well as in GUI executions) cannot be observed during execution at a lower level than the first level remote runbook, while for local runbook nesting it is visible all the time at any level down.
  - This step type can also be configured with Runbook Parameters, which can be referenced inside its runbook steps with the "$RP:" notation. The Runbook Parameters must also be defined inside the runbook yaml file. The values of these parameters defined in the runbook step will overwrite the values defined in the standalone runbook yaml file. They can contain references to other variables (e.g. to $OV, $SV or $PV), but they must not be mixed strings. Only single references are permitted.

4.5.2 Variable and item referencing in the runbook files

Given that the runbook files are yaml text files, assigning credentials in general cannot be done from this text file alone. For any step types, this is done by referencing in the step parameters the name of a credential from the local Secret Vault. But the referencing of secrets is not the only one. This referencing feature has been elaborated to mitigate the limitations of defining the parameters for entire orchestrations only from yaml text files, which would only allow string parameters.

Inside the SPOT runbooks, the step and command parameters can reference other variable objects as follows:

- Using the prefix "$OV:" will replace the entire parameter with the referenced OrchVars variable.
- Using the prefix "$OV:" and the suffix ":$OV" will replace the delimited string with the referenced OrchVars variable, which in this case must be a string.
- Using the prefix "$RP:" will replace the entire parameter with the referenced Runbook Parameter.

- Using the prefix “$RP:” and the suffix “:$RP” will replace the delimited string with the referenced Runbook Parameter, which in this case must be a string.
- A Runbook Parameter can also contain a “$RP:” reference, which in this case is for to the parent Runbook Parameters.
- Using the prefix “$PV:” will replace the entire parameter with the referenced PublishedData variable, any object type that may be (for remote published objects, they are available in $PublishedData only as de-serialized object types).
- Using the prefix “$PV:” and the suffix “:$PV” will replace the delimited string with the referenced PublishedData variable, which in this case must be a string.
- Using the prefix “$SV:” will replace the entire parameter with the referenced SVars secret variable, which originates in the local Secrets Vault.
- Using the prefix “$SV:” and the suffix “:$SV” will replace the delimited string with the referenced SVars secret variable, which in this case must be a string.
- All four types of variables, $RP, $OV, $PV and $SV, can be mixed in the same parameter as long as all of them reference strings. Also, array element referencing and object attribute addressing syntax can be used for such variables. For example, the following parameter with $PV and $OV referencing can be valid:
    - $PV:services[0].Name:$PVsome_padding$OV:ADConfig.DomainName
- The mixed string referencing is only permitted on Step Prameters and Command Parameters from Runbook Steps and not on Runbook Parameters and Runbook Remote Parameters.
- There is no need to reference the entire set of $OrchVars because they are already available all the time inside all step functions during execution, but the $PV and the $SV sets are not available in full all the time. To reference the entire $RP, $PV or $SV sets in a command parameter, the following syntax can be used, “$RP:.”, “$PV:.” or “$SV:.”.
- Using the prefix “$RFI:” will replace the relative path after the prefix with the absolute path to the referenced file from the current SPOT project folder, which must exist. For local steps, this is a local path, but for remote steps, this will be the path to the referenced file after it is made available on the target computer.
    - **The referenced file is copied** over to the remote computer temporarily, prior to the step execution. The file size is checked during runbook loading and if it is bigger than 10MB the runbook loading fails.
    - There is no suffix for this option, so the RFI reference must be the only element from a parameter.
    - Example: “$RFI:_Scripts\config.txt”
    - RFI stands for **R**EFERENCED **FI**LE.
- Using the prefix “$RFO:” will replace the relative path after the prefix with the absolute path to the referenced folder from the current SPOT project folder, which must exist (here there are two special cases “$RFO:SSHNET” and “$RFO:PSEXEC”, which are special references for the respective tools, from the SPOT Module folder). For local steps, this is a local path, but for remote steps, this will be the path to the referenced folder after it is made available on the target computer.
    - A folder can be referenced by its relative folder path or by using the relative path of a child file. In this way, if a file is referenced like this, the file referenced will be made available on the target computer along with its parent folder. This is useful if a step

function has as parameter a path to an executable that has dependencies in its own folder.
    - **The referenced folder is copied** over to the remote computer temporarily, prior to the step execution. The size is checked during runbook loading only if a file is referenced and if it is bigger than 10MB the runbook loading fails.
    - There is no suffix for this option, so the RFO reference must be the only element from a parameter.
    - Examples: "$RFO:_Scripts\config.txt" (folder referenced with file path) or "$RFO:tools" (folder referenced with folder path) or "$RFO:" (the entire project folder referenced)
    - RFO stands for **R**EFERENCED **FO**LDER.
- "$RFI:" and "$RFO:" reference an existing local file or folder, mainly from the project folder (exceptions are $RFO:SSHNET and $RFO:PSEXEC, which are located in the SPOT Module). For referencing paths for files that do not exist initially, like export file paths, the $OrchVars._ProjectPath variable can be used as a prefix to make sure the file will be placed inside the Project folder, wherever this may be. For example, the following notation can be used: "$OV:_ProjectPath:$OV\Files\export.csv". This is only applicable to locally executed Steps.
- When referencing files/folders from inside the Project folders, it is important not to reference files/folder from inside the following project sub-folders, as they are excluded from transport to other computers in case of remote runbook executions (their content is not needed or sensitive for remote exposure):
    - __SPOT_Config
    - __SPOT_Runbooks
    - __SPOT_Artefacts
    - _HelperFunctions

4.5.3 Creating the runbook control files

Similar to the OrchVars.yaml and SInputs.yaml files, it is recommended to create and edit the runbook files also with a suitable text editor, such as Notepad++, as they are also in the yaml format. The easiest way to learn is with explained examples that contain all possible parameters. For easy creation and editing of runbook control files, a full set of example step definitions is included by default in all newly create SPOT projects. The file is called StepTemplates.yaml and is located in the root of the project folder. Inside it, all possible SPOT parameters are present for all step types as well as minimal samples, only with the mandatory fields.

PowerShellCommandLocal sample step definition, with all possible parameters set:

```yaml
Step1:
    Type: PowershellCommandLocal
    Description: full example for PowershellCommandLocal
    Seq: 1
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFO
        CommandParameters:
            ToolPath: $RFO:Tools\curl\curl.exe
        ExecPath: C:\temp
        VariablesToPublish:
        - oneRFO=RemoteOne
        - twoRFO
    Disabled: false
```

- Type represents the method of execution. In this case, it is a local PowerShell execution. This element is mandatory on all step types.
- Description is self-explanatory. It is not mandatory.
- Seq must be an integer and represents the sequence number, which is used to determine the order of steps inside a sequence and if multiple steps are to be executed in parallel. The order of appearance of the steps in the yaml file is not considered. This element is mandatory on all step types.
- Disabled is a flag that can set a step to be ignored. This element is not mandatory and when is missing, the default value is False.
- ContinueOnError is a flag that can be used to set/override the behavior of the sequence when the current step returns error. This element is not mandatory and when is missing, the value of the OrchVar project setting "_ContinueOnError" is used. If that OrchVar project setting is not specified, it is created automatically with the value False.
- Conditions are a set of variables that can be referenced from OrchVars or Published Variables and that are used to determine if the current step is to be executed or skipped. The evaluation is done right before execution, to be able to evaluate any potential Published Variables from before. This element is not mandatory. For the conditions to be evaluated as ok to proceed, the variables configured as conditions for a step must **not** be empty ($null or “”) or False. In case the conditions are not all evaluated as ok to proceed, the respective Runbook step is skipped and the next steps from the sequence are processed.
- RetryCount must be an integer and represents the maximum number of retries that SPOT will attempt in case of step errors. This element is not mandatory and when is missing, the value of the OrchVar project setting "_RetryCount" is used. If that OrchVar project setting is not specified, it is created automatically with the value 0.
- RetryDelay must be an integer and represents the number of seconds that SPOT will wait between retry attempts, in case of step errors. This element is not mandatory and when is missing, the value of the OrchVar project setting "_RetryDelay" is used. If that OrchVar project setting is not specified, it is created automatically with the value 30.

- CommandName represents the name of the step function to be executed. Any function or script defined in the project, as well as any SPOT Built-in Step Function can be used here. This element is mandatory.
- CommandParameters are a set of elements representing the parameters needed for the step function (the one set in the CommandName parameter). As long as the step function has mandatory parameters, corresponding elements must be present here.
- VariablesToPublish are a set of elements representing variable names from inside the step function. The last values from those variables at the end of the execution are taken and published in a common HashTable called $PublishedData and made available to other steps for referencing. This element is not mandatory. If the variable needs to be published under a different name to avoid multiple steps publishing the same step function variable with the same name, the respective entry of the VariablesToPublish collection shall specify both the step function variable name and the new variable name separated by the equal symbol "=". (e.g. VarName=VarNameStepX).
- ExecPath represents the desired execution path for the step function. If the provided execution path is not found, a WARNING message is logged and the execution continues from the current location, wherever it might be. This is not a mandatory element.

PowerShellCommandRemote sample step definition, with all possible parameters set:

```
Step2:
    Type: PowershellCommandRemote
    Description: full example for PowershellCommandRemote
    Seq: 2
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFI
        CommandParameters:
            CSVFilePath: $RFI:Files\configs.csv
        RemoteComputer: 192.168.99.20
        Credential: $SV:SrvCred2
        ExecPath: C:\temp
        VariablesToPublish:
        - oneRFI
        - twoRFI=RemoteTwo
        UseSSL: true
    Disabled: false
```

All elements from PowerShellCommandLocal are also applicable here. In addition:

- RemoteComputer represents the hostname/IP Address of the target computer, where to execute the step function. This element is mandatory.
- Credential represents the credential object to be used to connect and execute the step function remotely. This element is mandatory.
- UseSSL can be used when the target computer is configured with an HTTPS WinRM listener, in order to encrypt the communication. This element is not mandatory.

PowerShellCommandRemoteSJ sample step definition, with all possible parameters set:

```
Step3:
    Type: PowershellCommandRemoteSJ
    Description: full example for PowershellCommandRemoteSJ
    Seq: 3
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFO
        CommandParameters:
            ToolPath: $RFO:Tools\curl\curl.exe
        RemoteComputer: 192.168.99.20
        Credential: $SV:SrvCred2
        ExecPath: C:\temp
        VariablesToPublish: TestVarName
        UseSSL: true
        # AsUser: $SV:SrvCred
        # AsSystem: true
    Disabled: false
```

All elements from PowerShellCommandRemote are also applicable here. In addition:

- AsUser represents the user credentials to be used to execute the step function remotely, which can be different from the credentials used to connect to the remote computer. This element is not mandatory. When not specified, the same credentials used for the remote connection are also used for the execution.
- AsSystem represents a flag to signal that the step function should be executed under the local system account. This element is not mandatory and not compatible with the AsUser element (they should not be specified together in a step).

PowerShellCommandRemoteWMI sample step definition, with all possible parameters set:

```
Step4:
    Type: PowershellCommandRemoteWMI
    Description: full example for PowershellCommandRemoteWMI
    Seq: 4
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFI
        CommandParameters:
            CSVFilePath: $RFI:Files\configs.csv
        RemoteComputer: 192.168.99.20
        Credential: $SV:SrvCred2
        ExecPath: C:\temp
        VariablesToPublish: TestVarName=TestVarName4
    Disabled: false
```

All elements from PowerShellCommandRemote are also applicable here.

PowerShellCommandRemoteOWMI sample step definition, with all possible parameters set:

```yaml
Step6:
    Type: PowershellCommandRemoteOWMI
    Description: full example for PowershellCommandRemoteOWMI
    Seq: 6
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFI
        CommandParameters:
            CSVFilePath: $RFI:Files\configs.csv
        RemoteComputer: 192.168.99.20
        Credential: $SV:SrvCred2
        ExecPath: C:\temp
        VariablesToPublish: TestVarName=TestVarName6
    Disabled: false
```

All elements from PowerShellCommandRemote are also applicable here.

PowerShellCommandRemotePsExec sample step definition, with all possible parameters set:

```yaml
Step5:
    Type: PowershellCommandRemotePsExec
    Description: full example for PowershellCommandRemotePsExec
    Seq: 5
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFO
        CommandParameters:
            ToolPath: $RFO:Tools\curl\curl.exe
        RemoteComputer: 192.168.99.20
        Credential: $SV:SrvCred2
        ExecPath: C:\temp
        VariablesToPublish: TestVarName=TestVarName5
        # AsSystem: true
    Disabled: false
```

All elements from PowerShellCommandRemote are also applicable here. In addition:

- AsSystem represents a flag to signal that the step function should be executed under the local system account. This element is not mandatory.

Runbook sample step definition, with all possible parameters set:

```
SingleLineTest3:
    Type: Runbook
    RunbookName: SingleLineTest
    Description: full example for runbook using RPs
    RunbookParameters:
        Par1: $RP:TopRPar2
        Par2: $PV:RemoteRParTwo
    Seq: 3
    Conditions: $OV:ConditionFlag
    ContinueOnError: false
    RemoteParameters:
        ExecFunction: PowershellCommandRemote
        RemoteComputer: 192.168.99.20
        Credential: $SV:SrvCred2
        UseSSL: true
        # AsSystem: true
    Disabled: false
```

All first level elements from PowerShellCommand types are also applicable here, with the exception of RetryCount and RetryDelay. That is becasue there is no retry possibility for entire runbooks, only for runbook steps.

The following specific elements can be defined:

- RemoteParameters > ExecFunction represents the step type function to be used for the remote runbook executions (can be: PowershellCommandRemote, PowershellCommandRemoteSJ, PowershellCommandRemoteWMI, PowershellCommandRemoteOWMI or PowershellCommandRemotePsExec).
- RemoteParameters > RemoteComputer represents the hostname/IP Address of the target computer to be used as a reply and execute the current runbook.
- RemoteParameters > Credential represents the PSCredential to be used for connecting and executing the current runbook remotely.
- RemoteParameters > UseSSL set to true represents the option to use WinRM over HTTPS for the ExecFunction. This is only applicable to the PowershellCommandRemote and PowershellCommandRemoteSJ exec functions.
- RemoteParameters > AsSystem set to true represents the option to execute the runbook remotely using the local system account from the remote computer. This is only applicable to the PowershellCommandRemoteSJ and PowershellCommandRemotePsExec functions.
- RunbookParameters are user defined parameters and they must be also defined in the called child runbook. The values defined here overwrite the values defined in the standalone child runbook, similarly with the parameters of a function being defined at invocation and overwriting any default parameter values. These Runbook Parameters can also reference parent Runbook Parameters or any other SPOT Vars, like $OV, $SV or $PV. The limitation is that no mixed string references are permitted.

These parameter groups are not mandatory. If the RemoteParameters are is not defined, the runbook will be executed from the local SPOT computer. If the RunbookParameters are not defined, the runbook will be executed with its own locally defined Runbook Parameters.

4.5.4 Steps with implied parallel execution

As mentioned earlier, the remote steps can be configured to execute on several targets without having a dedicated step for all of them. This is called implied parallel execution and can be achieved by setting multiple values in the “RemoteComputer” field from the respective step parameters. Because there is a single step configuration for this parallel execution, all other step parameters remain the same, including the credential object used for remote authentication.

When this mode of execution is used, the artefacts generated by the step are separated for each target computer with the addition of the suffix “-%TargetComputer%” in the file names and in the PublishedData hashtable keys that point to the published variable values (which are assimilated as published variable names).

Here are two samples for runbook steps configured for implied parallel execution.

```
Step2:
    Type: PowershellCommandRemote
    Description: full example for PowershellCommandRemote
    Seq: 2
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFI
        CommandParameters:
            CSVFilePath: $RFI:Files\configs.csv
        RemoteComputer: 192.168.0.20,192.168.0.21,192.168.0.22
        Credential: $SV:SrvCred2
        ExecPath: C:\temp
        VariablesToPublish:
        - oneRFI
        - twoRFI=RemoteTwo
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description: full example for PowershellCommandRemote
    Seq: 2
    Conditions: $OV:cFlag
    RetryCount: 0
    RetryDelay: 40
    ContinueOnError: false
    StepParameters:
        CommandName: TestFunctionRFI
        CommandParameters:
            CSVFilePath: $RFI:Files\configs.csv
        RemoteComputer:
        - 192.168.0.20
        - 192.168.0.21
        - 192.168.0.22
        Credential: $SV:SrvCred2
        ExecPath: C:\temp
        VariablesToPublish:
        - oneRFI
        - twoRFI=RemoteTwo
    Disabled: false
```

As seen above, the VariablesToPublish values remain the same in the runbook step but they are generated separately by SPOT automatically.

4.5.5 Using SPOT built-in step functions

When creating runbooks, besides the user-made step scripts and functions there are also available a limited number of SPOT built-in step functions. They should be regarded as both examples to how step functions should look like as well as helpful building blocks for user-made runbooks. Some of them are very simple while others are very complex and powerful. Most of them rely on the SSHNet tool which is included in SPOT as an extension, to allow flexible orchestration of Linux based target computers and network appliances.

To use them, simply identify a function of interest and use its name as a step function (step parameter "CommandName") along with any needed command parameters. Some of them are not only suitable as step functions but also as helper functions. They are not exposed from the SPOT module but they can be inspected in place as they are located in a separate file inside the SPOT module called SPOTStepFunctions.ps1. They are documented based on PowerShell guidelines but since they are not exposed outside the SPOT module, their Help is not available unless the file is inspected directly or loaded manually in a PowerShell prompt.

Below is the full list of SPOT built-in step functions with their description and runbook step samples.

- **Set-FileExecutableSFTP**
  - Connects remotely to a Linux computer over SFTP, checks the existence of the target file then adds execute permissions for everyone.
  - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSFTPSession.

```yaml
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Set-FileExecutableSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            FilePath: /root/test.sh
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```yaml
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Set-FileExecutableSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            FilePath: /root/test.sh
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Download-FileSFTP**
  - Connects remotely to a Linux computer over SFTP, checks the existence of the target file then downloads it locally if the file is not already present locally.
  - The local file existence check is based on file size.
  - If the parent folder of the local path does not exit, it will be created.
  - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSFTPSession.

```
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Download-FileSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/test.csv
            LocalPath: $OV:_ProjectPath:$OV\Tools\test.csv
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Download-FileSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/test.csv
            LocalPath: $OV:_ProjectPath:$OV\Files\test.csv
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Upload-FileSFTP**
  - Checks the local file exists, connects remotely to a Linux computer over SFTP, checks the target file path and if does not exist it uploads it remotely.
  - The remote file existence check is based on file size.
  - If the parent folder of the remote path does not exit, it will be created.
  - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSFTPSession.

```
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Upload-FileSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/test.sh
            LocalPath: $OV:_ProjectPath:$OV\Tools\test.sh
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Upload-FileSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/test.sh
            LocalPath: $RFI:Files\test.sh
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Download-FolderSFTP**
  - Connects remotely to a Linux computer over SFTP, checks the existence of the target folder then downloads its content locally if the folder content is not already present locally.
  - The local folder content existence check is based on file size check for every content file.
  - If the target local folder path does not exit, it will be created.
  - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSFTPSession.

```
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Download-FolderSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/curl
            LocalPath: $OV:_ProjectPath:$OV\Tools\curl
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Download-FolderSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/test
            LocalPath: $OV:_ProjectPath:$OV\Files\test
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Upload-FolderSFTP**
  - Checks the local folder exists, connects remotely to a Linux computer over SFTP and uploads the local folder content.
  - The remote folder content existence check is based on file size check for every content file.
  - If the folder of the remote path does not exit, it will be created.
  - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSFTPSession.

```
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Upload-FolderSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/curl
            LocalPath: $OV:_ProjectPath:$OV\Tools\curl
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Upload-FolderSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            RemotePath: /root/test
            LocalPath: $RFO:Files\test
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Test-FilePathSFTP**
  - Connects remotely to a Linux computer over SFTP and checks the existence of the target file.
  - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSFTPSession.

```yaml
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Test-FilePathSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            FilePath: /root/test.sh
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```yaml
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Test-FilePathSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            FilePath: /root/test.sh
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Test-FolderPathSFTP**
  - Connects remotely to a Linux computer over SFTP and checks the existence of the target folder.
  - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSFTPSession.

```
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Test-FolderPathSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            FolderPath: /root/test
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Test-FolderPathSFTP
        CommandParameters:
            TargetIP: 192.168.0.20
            FolderPath: /root/test
            Port: 22
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Execute-CMDScript**
  - Executes a CMD script with the parameters provided.
  - If input data is also provided, it monitors the output and sends the input value when the output line is matching the prompt match string. After the output string matching, there must be around 2 seconds of no other script output received to be considered a prompt for a value.
  - In the example below, under the “InputData” CommandParameter, the value pair “username%_%john” means that when the output prompt “username” appears, the value “john” should be sent to the script.
  - The file from the script path must allow renaming (not to be a ReadOnly path) if the script extension is not cmd or bat.
  - The $_spot_CMDOutput variable contains the full output.
  - It depends on the SPOT function Write-SPOTLog.

```
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-CMDScript
        CommandParameters:
            ScriptPath: $RFI:CMDScripts\test.cmd
            ScriptParameters: Par1 Par2
            InputData:
            - username%_%john
            - age%_%25
            Timeout: 180
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_CMDOutput=CMDOutputStep1
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-CMDScript
        CommandParameters:
            ScriptPath: $RFI:CMDScripts\test.cmd
            ScriptParameters: Par1 Par2
            InputData:
            - username%_%john
            - age%_%25
            Timeout: 180
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_CMDOutput=CMDOutputStep2
    Disabled: false
```

- **Execute-BashScript**
    - Executes a Bash script on a remote Linux computer, over SSH, with the parameters provided.
    - If input data is also provided, it monitors the output and sends the input value when the output line is matching the prompt match string. After the output string matching, there must be around 2 seconds of no other script output received to be considered a prompt for a value.
    - In the example below, under the “InputData” CommandParameter, the value pair “username%_%john” means that when the output prompt “username” appears, the value “john” should be sent to the script.
    - The initial prompt is detected automatically and there is a 10 seconds timeout for it.
    - The script must exist on the target computer or must be specified also locally, in order to be copied over before execution.
    - The parameter ElevationCommand specifies the command to be used to obtain elevation for the user specified in the Credential parameter. If this parameter is specified, the elevation will be attempted.
    - The parameter ElevationPassword specifies the password needed for the elevation. It is only used if the ElevationCommand parameter is set. If the ElevationPassword is not specified, the password of the user from the Credential parameter will be attempted. If the ElevationPassword parameter is specified, it will be used for elevation. If a null ElevationPassword is specified, no password will be sent after the elevation command.
    - IMPORTANT: any variables defined inside the bash script can be exported with the normal SPOT variable publishing mechanism simply by specifying them in the “VariablesToPublish” parameter of the runbook step.
    - The exit value of the bash script is honored (if the “exit” command is used inside the bash script or not). The “exit” command will be intercepted. If the exit value of the bash script has a non-zero value a terminating error will be triggered to signal the step with error.
    - The $_spot_BashOutput variable contains the full output, including the sent commands and the initial output.
    - To reference the SshNet tool for the $SshNetPath parameter in remote executions, the syntax "$RFO:SSHNET" can be used in the command parameter part in the SPOT runbook file. For local executed steps, the parameter “SshNetPath” does not need to be specified as it will be detected automatically by the Step function.
    - It depends on the SPOT functions Write-SPOTLog, Test-SPOTTCPPort, Get-SPOTSshNetPath, Upload-FileSFTP and New-SPOTSSHSession.

```yaml
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-BashScript
        CommandParameters:
            TargetIP: 192.168.0.20
            RemoteScriptPath: /root/test.sh
            LocalScriptPath: $RFI:BASHScripts\test.sh
            ScriptParameters: Par1 Par2
            InputData:
            - username%_%john
            - age%_%25
            Port: 22
            Timeout: 10
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            ElevationCommand: sudo
            ElevationPassword: $SV:SudoPass
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_FullOutput=OutputStep1
        - _spot_CmdOutputOnly=COOStep1
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```yaml
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-BashScript
        CommandParameters:
            TargetIP: 192.168.0.20
            RemoteScriptPath: /root/test.sh
            LocalScriptPath: $RFI:BASHScripts\test.sh
            ScriptParameters: Par1 Par2
            InputData:
            - username%_%john
            - age%_%25
            Port: 22
            Timeout: 10
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
            ElevationCommand: sudo
            ElevationPassword: $SV:SudoPass
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_FullOutput=OutputStep2
        - _spot_CmdOutputOnly=COOStep2
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Execute-SSHScript**
    - Connects to a remote device over SSH and executes a series of commands loaded from a local script file.
    - Before executing the commands from the local script file, each line is checked for $RP, $OV, $SV or $PV references and they get replaced, in string only mode.
    - Any secrets replaced in the commands may be visible in the log files.
    - To be able to replace a referenced variable, the entire set of Vars of that type must be made available to this step function with the exception of OrchVars, which are available by default inside all step functions.
        - For $RP references inside the SSH script, the $RPars parameter must be present in the command parameters for that step.
        - You can specify the entire set of Runbook Parameters with the notation: $RP:.
        - For $SV references use the parameter $SVars with the notation: $SV:.
        - For $PV references use the parameter $PVars with the notation: $PV:.
        - This is applicable to all Execute-SSHScript step types, local or remote.
        - If a certain variable type is not referenced in the SSH script, the respective parameter does not have to be specified.
    - To reference the SshNet tool for the $SshNetPath parameter in remote executions, the syntax "$RFO:SSHNET" must be used in the command parameter part in the SPOT runbook file.
    - If the remote host SSH keys must be checked before trusting, a file with the SSH key information must be specified with the TrustedHostsFilePath parameter. This enables the SSH remote host key verification by default.
    - Each line may contain a prompt matching string at the beginning, split with "%_%" from the actual command. This is for the prompt after the command has been executed.
    - The Timeout parameter is for lines without prompt matching string, to move on to reading the output after a command has been sent.
    - The ExpectTimeout parameter is for lines with prompt matching string, to move on to reading the output after a command has been sent and no prompt match has been made.
    - The initial prompt is considered any output read at the beginning after a 5 seconds wait time.
    - The parameter ElevationCommand specifies the command to be used to obtain elevation for the user specified in the Credential parameter. If this parameter is specified, the elevation will be attempted.
    - The parameter ElevationPassword specifies the password needed for the elevation. It is only used if the ElevationCommand parameter is set. If the ElevationPassword is not specified, the password of the user from the Credential parameter will be attempted. If the ElevationPassword parameter is specified, it will be used for elevation. If a null ElevationPassword is specified, no password will be sent after the elevation command.
    - The $_spot_FullOutput variable contains the full output, including the sent commands and the initial output.
    - The $_spot_CmdOutputOnly variable contains only the command outputs from all lines.

- The main purpose for this function is to be a step function and not really a helper function.
- It depends on the SPOT functions Write-SPOTLog, Test-SPOTTCPPort, Get-SPOTSshNetPath, New-SPOTSSHSession and Replace-SPOTLineVars.

```
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-SSHScript
        CommandParameters:
            TargetIP: 192.168.0.20
            ScriptPath: $RFI:SSHScripts\test.txt
            Port: 22
            Timeout: 10
            ExpectTimeout: 20
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            ElevationCommand: sudo
            ElevationPassword: $SV:SudoPass
            SVars: $SV:.
            PVars: $PV:.
            RPars: $RP:.
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_FullOutput=OutputStep1
        - _spot_CmdOutputOnly=COOStep1
        - _spot_SSH_Key=SSH_Key_Details_Local
    Disabled: false
```

```yaml
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-SSHScript
        CommandParameters:
            TargetIP: 192.168.0.20
            ScriptPath: $RFI:SSHScripts\test.txt
            Port: 22
            Timeout: 10
            ExpectTimeout: 20
            Credential: $SV:SSHCred
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
            ElevationCommand: sudo
            ElevationPassword: $SV:SudoPass
            SVars: $SV:.
            PVars: $PV:.
            RPars: $RP:.
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_FullOutput=OutputStep2
        - _spot_CmdOutputOnly=COOStep2
        - _spot_SSH_Key=SSH_Key_Details_Remote
    Disabled: false
```

- **Execute-TelnetScript**
    - Connects to a remote device over Telnet and executes a series of commands loaded from a local script file.
    - Before executing the commands from the local script file, each line is checked for $Cred, $RP, $OV, $SV or $PV references and they get replaced, in string only mode.
    - For Telnet the authentication is performed during script execution so the username and password must be referenced inside the script with $Cred:Username and $Cred:Password. These will be replaced with the credential elements from the $Credential function parameter.
    - Any secrets replaced in the commands may be visible in the log files.
    - To be able to replace a referenced variable, the entire set of Vars of that type must be made available to this function with the exception of OrchVars, which are available by default inside all step functions.
        - For $RP references inside the telnet script, the $RPars parameter must be present and specify the entire set of Runbook Parameters with the notation: $RP:.
        - For $SV references the parameter $SVars with the notation: $SV:.
        - For $PV references the parameter $PVars with the notation: $PV:.
        - This is applicable to all step types, local or remote.

        - If a certain variable type is not referenced in the SSH script, the respective parameter does not have to be specified.
    - Each line may contain a prompt matching string at the beginning, split with "%_%" from the actual command. This is for the prompt after the command has been executed.
    - The Timeout parameter is for lines without prompt matching string, to move on to reading the output after a command has been sent.
    - The ExpectTimeout parameter is for lines with prompt matching string, to move on to reading the output after a command has been sent and no prompt match has been made.
    - The initial prompt is considered any output read at the beginning after a 5 seconds wait time.
    - The $_spotFullOutput variable contains the full output, including the sent commands and the initial output.
    - The $_spot_CmdOutputOnly variable contains only the command outputs from all lines.
    - The main purpose for this function is to be a step function and not really a helper function.
    - It depends on the SPOT functions Write-SPOTLog, Test-SPOTTCPPort, Replace-SPOTLineCred and Replace-SPOTLineVars.

```yaml
Step1:
    Type: PowershellCommandLocal
    Description:
    Seq: 1
    Conditions: $OV:Step1Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-TelnetScript
        CommandParameters:
            TargetIP: 192.168.0.20
            ScriptPath: $RFI:TelnetScripts\test.txt
            Port: 23
            Timeout: 10
            ExpectTimeout: 20
            Credential: $SV:TelnetCred
            SVars: $SV:.
            PVars: $PV:.
            RPars: $RP:.
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_FullOutput=OutputStep1
        - _spot_CmdOutputOnly=COOStep1
    Disabled: false
```

```
Step2:
    Type: PowershellCommandRemote
    Description:
    Seq: 2
    Conditions: $OV:Step2Flag
    RetryCount: 0
    RetryDelay: 30
    ContinueOnError: false
    StepParameters:
        CommandName: Execute-TelnetScript
        CommandParameters:
            TargetIP: 192.168.0.20
            ScriptPath: $RFI:TelnetScripts\test.txt
            Port: 23
            Timeout: 10
            ExpectTimeout: 20
            Credential: $SV:TelnetCred
            SVars: $SV:.
            PVars: $PV:.
            RPars: $RP:.
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
        ExecPath: C:\temp
        VariablesToPublish:
        - _spot_FullOutput=OutputStep2
        - _spot_CmdOutputOnly=COOStep2
    Disabled: false
```

- **Reboot-LinuxComputer**
    - Connects to the target computer over SSH and triggers a reboot of the computer, after which it waits for the SSH port to be available again and checks the startup readiness.
    - Both the trigger of the reboot and the startup readiness check are performed with remotely copied bash scripts.
    - The startup readiness check is done using the "systemctl is-system-running" command or testing the "runlevel", depending on the type of Linux. The type of linux startup system is detected automatically.
    - It depends on several SPOT functions, like Test-SPOTTCPPort, Write-SPOTLog, Get-SPOTSshNetPath and New-SPOTSSHSession.

```yaml
Step1:
    Type: PowershellCommandLocal
    Description: Reboot function for Linux
    Seq: 1
    StepParameters:
        CommandName: Reboot-LinuxComputer
        CommandParameters:
            TargetComputer: 192.168.99.30
            Credential: $SV:AdminSSH
            Port: 22
            Timeout: 300
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        VariablesToPublish:
        - _spot_FullOutput=FullRebootOutput
    ContinueOnError: true
    Disabled: false
```

```yaml
Step2:
    Type: PowershellCommandRemote
    Description: Reboot function for Linux
    Seq: 2
    StepParameters:
        CommandName: Reboot-LinuxComputer
        CommandParameters:
            TargetComputer: 192.168.99.30
            Credential: $SV:AdminSSH
            Port: 22
            Timeout: 300
            TrustedHostsFilePath: $RFI:Files\TrustedHosts.csv
            SshNetPath: $RFO:SSHNET
        VariablesToPublish:
        - _spot_FullOutput=FullRebootOutput
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
    ContinueOnError: true
    Disabled: false
```

- **Reboot-WindowsComputer**
    - Connects to the target computer over PSSession and triggers a reboot, after which it waits for the PSSession port to be available again and checks the startup readiness.
    - The startup readiness check is done by two conditions; either the LogonUI process exists, either there is an interactive user logged in. This is essentially a check that after reboot, Windows reached the point where the login screen appears, prompting for login credentials.
    - It supports the WinRM over HTTPS protocol.
    - It depends on several SPOT functions, like Test-SPOTTCPPort and Write-SPOTLog.

```
Step3:
    Type: PowershellCommandLocal
    Description: Reboot function for Windows
    Seq: 3
    StepParameters:
        CommandName: Reboot-WindowsComputer
        CommandParameters:
            TargetComputer: 192.168.99.20
            Credential: $SV:SrvCred2
            Timeout: 300
            UseSSL: true
    ContinueOnError: true
    Disabled: false
```

```
Step4:
    Type: PowershellCommandRemote
    Description: Reboot function for Windows
    Seq: 4
    StepParameters:
        CommandName: Reboot-WindowsComputer
        CommandParameters:
            TargetComputer: 192.168.99.20
            Credential: $SV:SrvCred2
            Timeout: 300
            UseSSL: true
        RemoteComputer: 192.168.0.2
        Credential: $SV:SrvCred
    ContinueOnError: true
    Disabled: false
```

The remote execution runbook step examples are for the situations where the SPOT computer itself cannot reach the target device or Linux computer and a second hop is needed to initiate the network connection and to execute the step function. The remote step type can be any of the remote powershell step types.

IMPORTANT NOTE: As seen in the pictures above, all SSH and SFTP related SPOT built-in functions also provide a parameter called "*TrustedHostsFilePath*". The use of this parameter enables a security feature in all these functions, the SSH Key validation. For initializing and adding entries in this Trusted Hosts file, a dedicated SPOT function is available "*Add-SPOTSSHTrustedHostKey*" which allows to add entries based on: SSH Key files (made available to the function via file paths), SSH Key values that can be lookup in the SPOT log during trusted (but with no SSH Key validation) SSH connections to the same target hosts or based on SPOT_SSH_Key objects which are generated and can be published for all SSH related steps.

In the SPOT Orchestration log, right after the SSH session is connected, a log line similar to the following is logged, which can be used to get the needed data and manually populate the Trusted Hosts file:

INFO: SSH server key details: Host=>xx.xx.xx.xx,Port=>22,Type=>ssh-ed25519,SHA256Fingerprint=>xxxx....

4.5.6 Common SPOT built-in variables

There are a few standardized internal SPOT variables, the ones that have the reserved prefix "_spot_", which can be used as published variables in subsequent steps.

- _spot_Output – which contains only the step function output. It is the same content that is added to the step log files after the initial logging of the SPOT step preparations.
- _spot_ExitCode – which contains the exit code of the step function, in case the exit command was used in the PowerShell step function and triggered. Not all step functions are expected to have exit statements, in which case this variable will be empty.

There are also a few internal SPOT variables specific to some of the SPOT built-in step functions. They are documented already above, for each step function, but here there is a good summary:

- _spot_ExitCode – appears here again because in the context of the built-in step functions (which do not have exit statements) it represents the exit code of the underlying script executed, either the CMD or the BASH script. It is applicable only to the following step functions:
    - Execute-CMDScript
    - Execute-BashScript
- _spot_CMDOutput – which contains only the CMD script output and not the extra output of the PowerShell step functions. It is applicable only to the following:
    - Execute-CMDScript
- _spot_BashOutput – which contains only the BASH script output and not the extra output of the PowerShell step functions. It is applicable only to the following:
    - Execute-BashScript
- _spot_FullOutput – which contains the entire underlying script output, the sent commands and the received output. It is applicable only to the following step functions:
    - Execute-SSHScript
    - Execute-TelnetScript
    - Reboot-LinuxComputer
- _spot_CmdOutputOnly – which contains only the output received for the commands sent by the underlying script. It is applicable only to the following step functions:
    - Execute-SSHScript
    - Execute-TelnetScript

4.5 Executing runbooks

First, it is important to remember that the SPOT project initializations are done not only per system but also per user. So a SPOT project can only be executed from the user account used for the initialization. This is because all SPOT projects have a single secret vault associated and the secret vaults can only be defined one per user. The registered state of the secret store master password has no impact on this topic.

Second, it is also recommended to use new PowerShell environments when executing SPOT Runbooks. In other words, the PowerShell command prompt used for a SPOT execution should not be re-used, but closed and a new one opened for a new execution. This is because some variables and runspaces may not be properly reinitialized and due to this they may alter the normal behavior of the SPOT execution.

Before a runbook is executed, it will first be loaded. With the GUI method this is done manually by selecting the target runbook and pressing a button to load it. With the command line method, this is done transparently after launching the command “Start-SPOT” with the target runbook name as a parameter. During the loading of a Runbook, besides the actual loading from yaml file into a PowerShell variable, there are several other things that happen for each step:

- The step definition is checked in order to find if all mandatory step parameters are actually defined in the step.
- The step function is checked for parameters and then the command parameters defined in the Runbook step are checked against them to make sure that no mandatory parameters are missing or that all parameters specified in the Runbook step are actually defined in the step function. If there are any parameters defined wrong the loading will stop with an error.
- The step parameters and command parameters are checked for $OV and $SV references and replaced. The only exception to this replacement is if the parameter contains more than one variable reference, making it a string only replacement with potential $PV references as well. In this case the reference replacement will happen during the execution of the runbook, just before the respective step is due.
- The command parameters are checked for $RFI and $RFO references and if found, the referenced file, where applicable, is checked for size in order not to exceed the 10MB limit.

When executing SPOT Runbooks, the executed Runbook is called also the MAIN Runbook, as it can contain child Runbooks on several levels and it is important to differentiate the top-most, which can be only one.

During the execution of the main runbook, each runbook step (which can be an actual RunbookStep or even a child Runbook) will cycle through a set of potential states:

- Initial – the step object has not been triggered for execution yet.
- Executing – the step object has begun execution and has not yet finished it.
- Error – the step object has encountered an error while launching or the payload returned a terminating error during execution.
- Skipped – the step object was skipped from execution after the associated conditions have been evaluated as such.

- Paused – the step object (in this case a runbook) has received a signal to stop/pause during execution and it is now in an uncompleted state.
- Disabled – the step object was marked from the loading phase as disabled, to prevent any processing of its referenced variables/conditions and any execution trigger.
- Completed – the step object has finished the execution phase without any terminating error.

4.5.1 Command line runbook execution

The full command to start a SPOT Runbook execution is:

Start-SPOT -ProjectPath %ProjectPath% -MainRunbookName %RunbookName% [-MasterPassword %MasterPassword%]

If the execution is started from the project folder, the ProjectPath parameter can be ".". This parameter is mandatory.

The MainRunbookName represents the base name (the file name without the .yaml extension) of the runbook yaml file. This parameter is mandatory.

The MasterPassword represents the secret store master password, in clear text. This parameter is not mandatory because the master password may be registered in the user environment during the SPOT initialization for that user.

After the execution of the SPOT Runbook in a PowerShell command prompt, the $OrchVars, $PublishedData, $SVars and the internal PowerShell objects of the Runbooks and steps are still available for inspection, in case a troubleshooting is needed. More details about these can be found later in this document.

During execution, the progress will be illustrated in the command line with periodic output messages.

4.5.2 GUI runbook execution

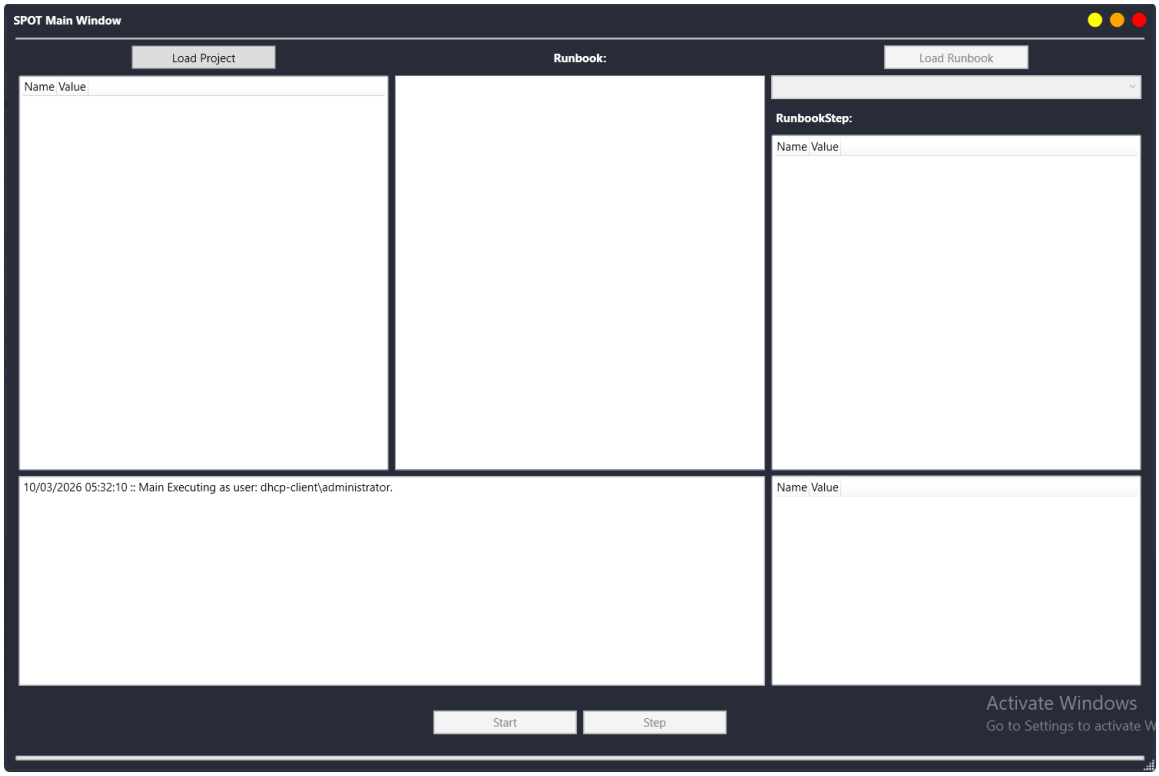
This method is best suited for assisted orchestrations, like deployments. Under the hood, the same functions are used to drive the orchestration. The difference is that they are wired into a GUI window. The main advantage for the GUI method is the possibility to stop and resume the execution. The resume is possible not only when the runbook execution is stopped manually, but also when the execution is stopped due to failure. In addition to this, the visualization of the runbook file structure and the progress report are much better.

There are no GUI methods to edit the runbook files. The GUI is made only for visualization of status and execution control.

To execute a SPOT runbook from a SPOT project with GUI, the following command can be used:

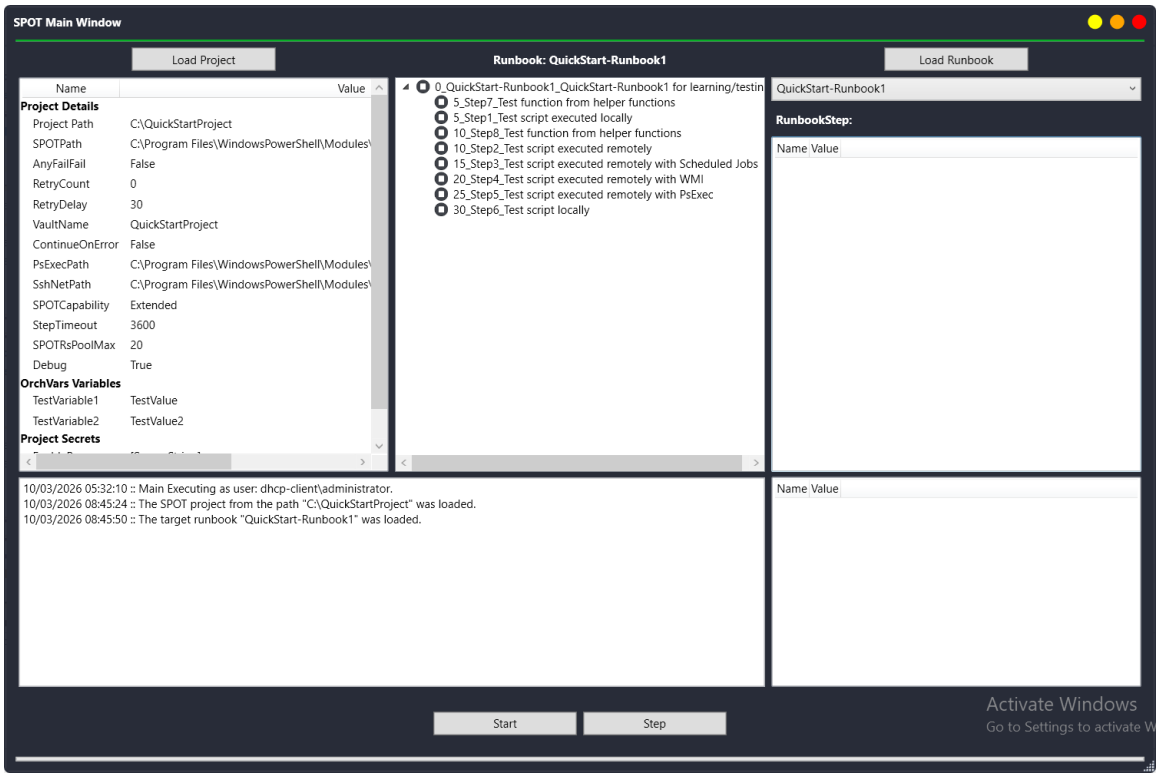
Start-SPOTGUI

The SPOT GUI opens empty and looks like in the picture below.



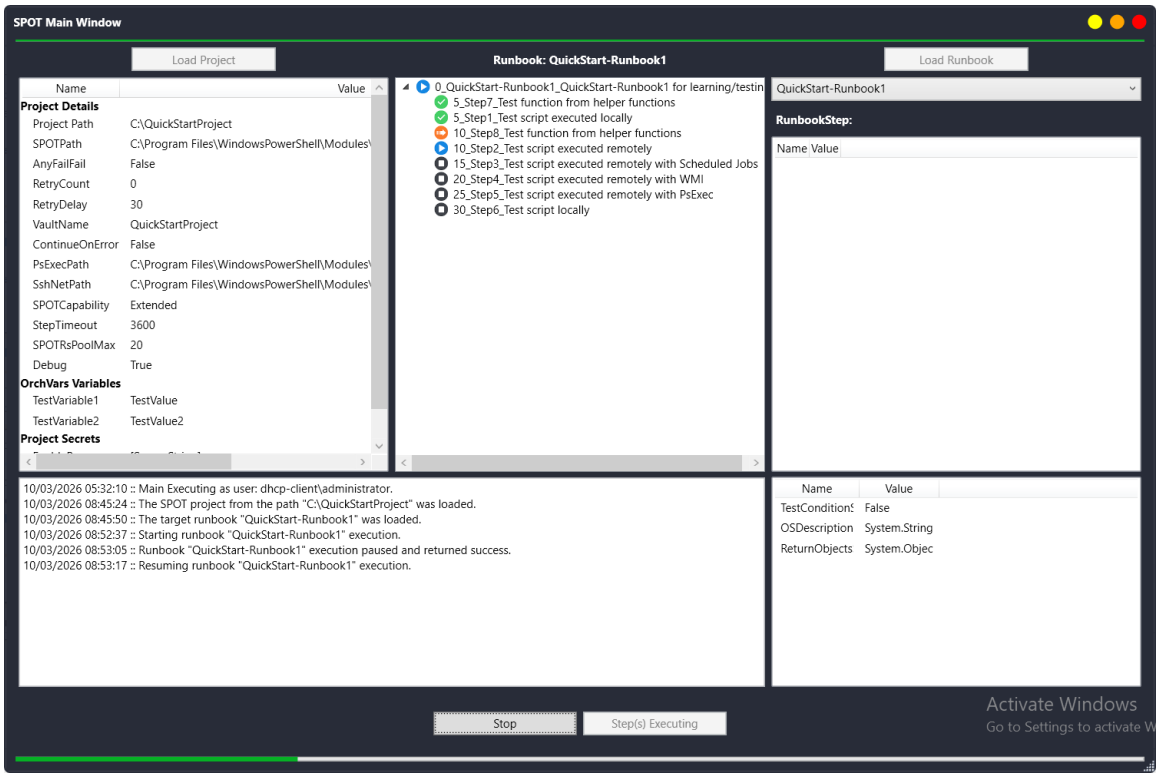
SPOT GUI – Initial screen

The target project can be loaded with the “Load Project” button and afterwards, the target runbook can be loaded with the “Load Runbook” button after selecting it from the drop-down menu below the button. The GUI will then reflect the main elements from the project as well as from the loaded runbook, as seen in the picture below.



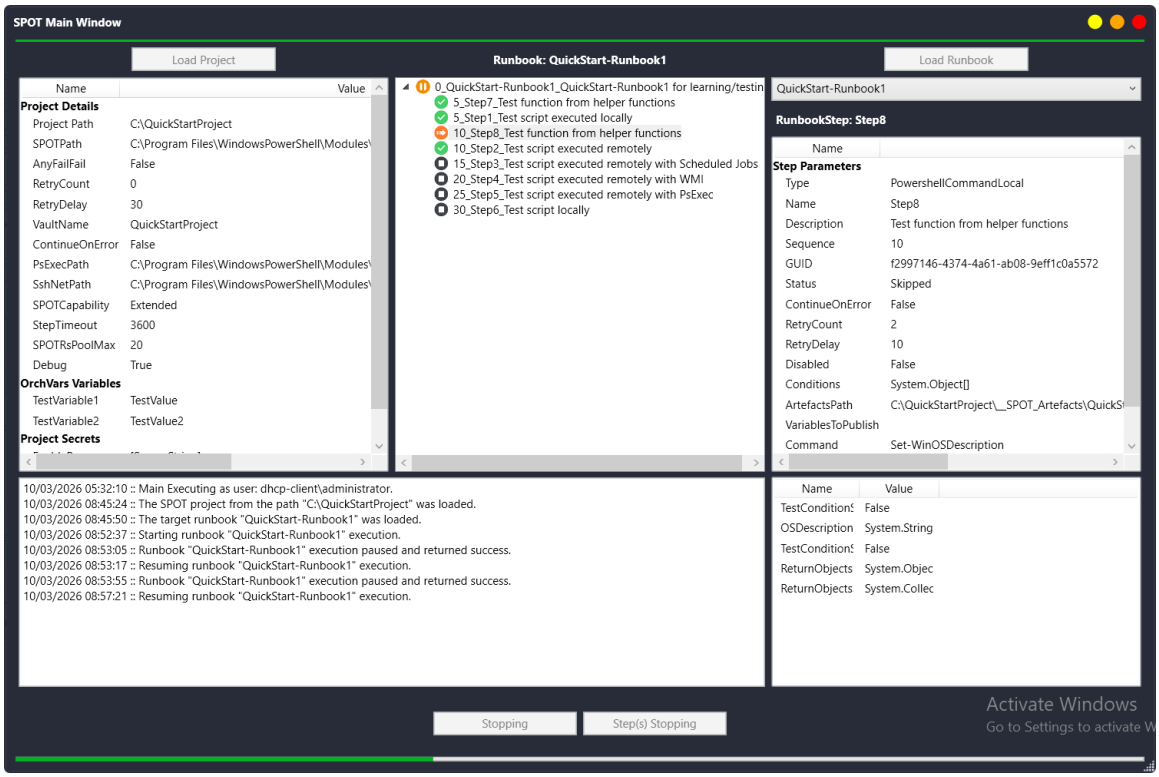
SPOT GUI – Loaded Project/Runbook

At this point, the runbook execution can be triggered by pressing the button “Start”. If the button “Step” is pressed, only the steps with the same next Seq number (that imply parallel execution) will be executed, allowing a step by step progressive runbook execution. This is useful in runbook development situations, when the effects of any step need to be studied before jumping to the next steps. This is how the GUI reflects the execution progress.



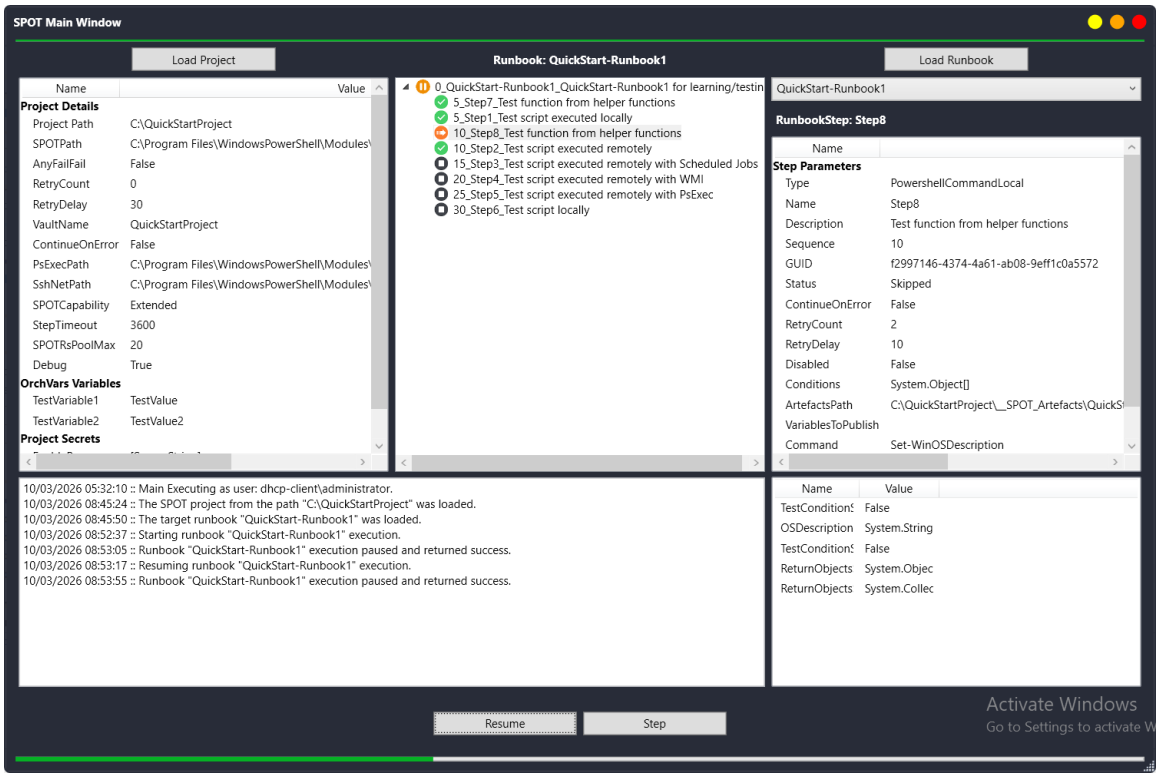
SPOT GUI – Executing Runbook

In the SPOT GUI there is the possibility of stopping/pausing the runbook execution by pressing the “Stop” button. The stop will be delayed until any currently executing steps are completed.



SPOT GUI – Stopping Runbook

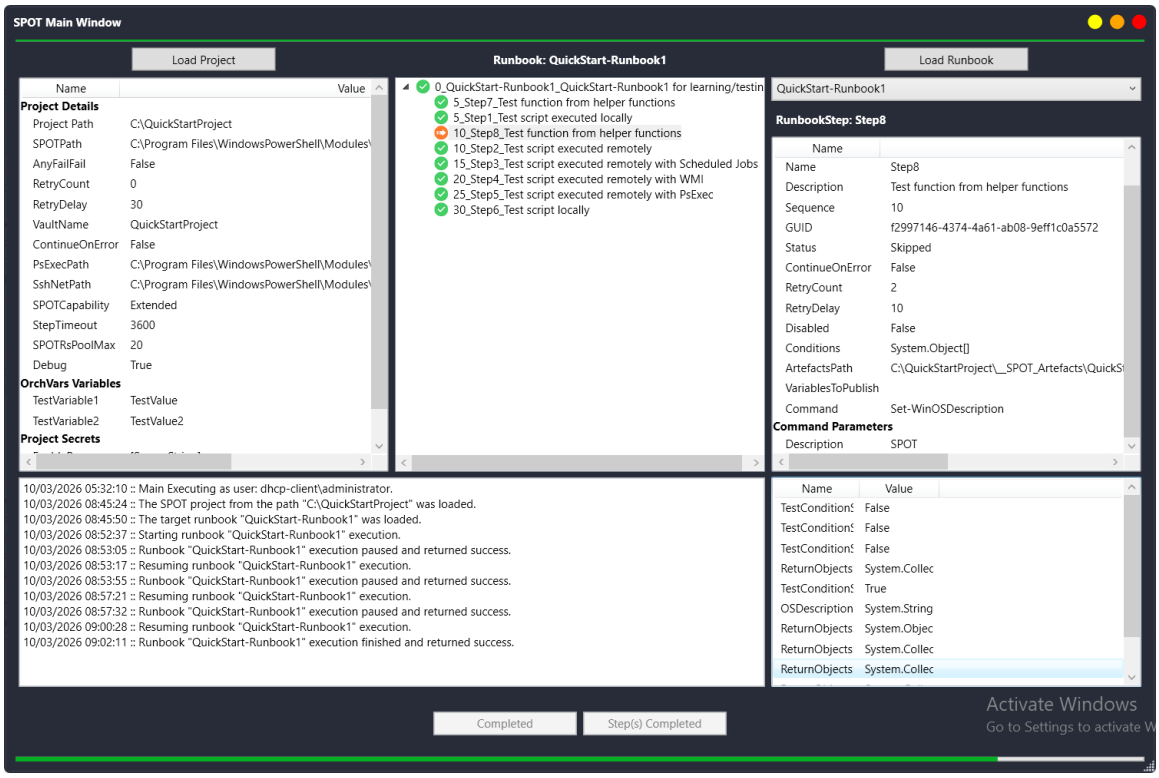
After the “Stopping” phase is completed, the GUI remains in a “Paused” status, from which it can be resumed.



SPOT GUI – Paused Runbook

The “Paused” runbook status is more visible at the runbook level because this status is not applicable to runbook steps.

And this is how it looks like at the end, if all steps were successful.



SPOT GUI – Completed Runbook

As seen in the last picture above, when selecting a runbook step, its details are available in the right side area of the GUI.

Also in the lower right side of the GUI, the PublishedData variables can be observed. If their type is string compatible or Boolean, the values can be seen in this GUI area. If not, only the object type is displayed.

When the main runbook loaded in the GUI contains child runbooks, the description text displayed is taken from the child runbook description field in the child runbook yaml file and not from the description assigned to the child runbook step in the parent runbook yaml file.

4.6 Artefacts

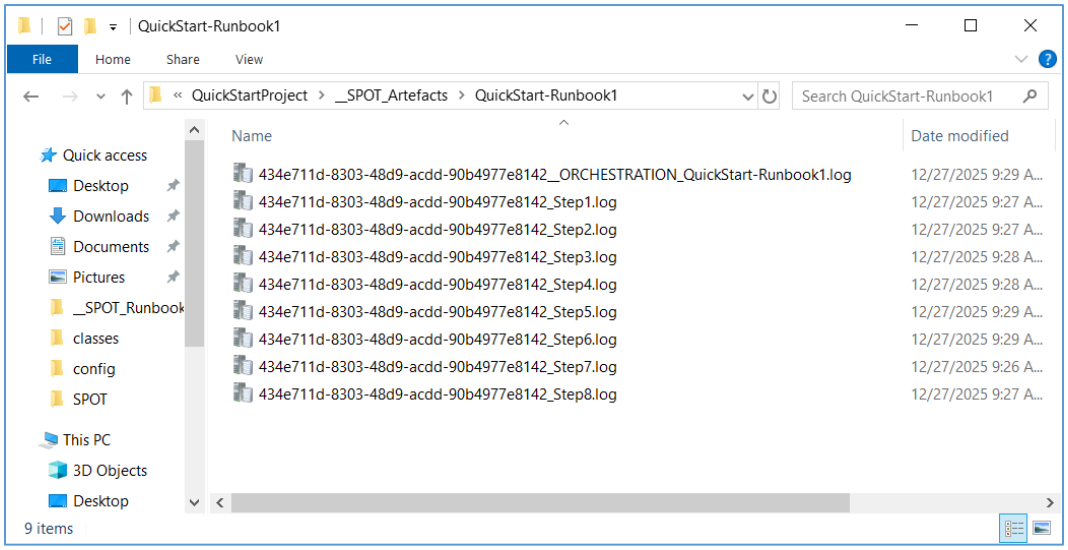
The SPOT tool itself has a default target log file that can provide information about what is going on internally inside the tool during the execution. This default log file can be found in "C:\Windows\temp\Orchestration.log" both locally and also on remote computers, when runbooks or runbook steps are executed remotely.

For the step functions, the logging to file functionality must be provided by the project functions, if desired. This is not mandatory in any way, but it is recommended, so that the orchestration could be

observed in real time, during execution. It is also recommended to avoid setting as target the same log file as the SPOT log file, which may be a little too crowded.

After the main runbook has finished the execution, there are two places to check for execution artefacts. **First location**, in the project folder "__SPOT_Artefacts" where all the output from each step (including warnings and errors) is present in its own log file and the files are organized in folders representing the runbooks they belong to. Each runbook has also a runbook execution log file that contains internal SPOT logging. **Second location**, inside the PowerShell window, the internal SPOT objects are still available, like, for example, the $PublishedData variable that contains all variables that have been published by all the steps in the orchestration.

In the picture below the resulting log file artefacts from a runbook execution can be seen. Each filename contains the internal SPOT GUID of the runbook they belong to and the step names.



It is important to note that the step output files are created only after the execution has finished, regardless if it finished successfully or with an error. When troubleshooting failed steps, this file can provide valuable information to understand where the execution failed ant what errors where returned, because the SPOT tool saves also the details of the error (both terminating and non-terminating) in this file.

4.6.1 Error handling

Sometimes, during a step function execution, errors may appear. Because the execution can happen locally or remote, with limited or no access to the error objects inside potential several layers of runspaces, SPOT provides some error handling to help identify where the error was triggered and to provide details about it. This is all provided via the artefact logs, which should contain all output from all step functions, including warnings and errors.

In PowerShell there are two main error types, non-terminating and terminating errors. This is how SPOT handles them:

Non-terminating errors

The error output generated by PowerShell is included, intertwined, in the normal output, so that it is easy to understand what part of the step function has triggered it based on the normal output before and after the error details.

This can be considered polluting the normal output but I think it is more important to have also the positioning information when an error occurs, to allow better understanding. In the end, the point is to avoid the errors, even the non-terminating ones, by changing the step function/script based on this error information.

Since the non-terminating errors do not stop the step execution, the output logs must be checked actively for these type of errors.

Terminating errors

When a terminating error occurs during a step function execution, the following message is logged "############## ORCHESTRATOR LOGGING: TERMINATING ERROR while trying to execute the step function […]. ##############"

In case of a terminating error encountered during the step function execution, the main purposes of the logging are to identify where the error came from and to provide enough details about it to help avoiding it in the future (either through catching it inside the step function, either through other code modifications). Here we have two main cases.

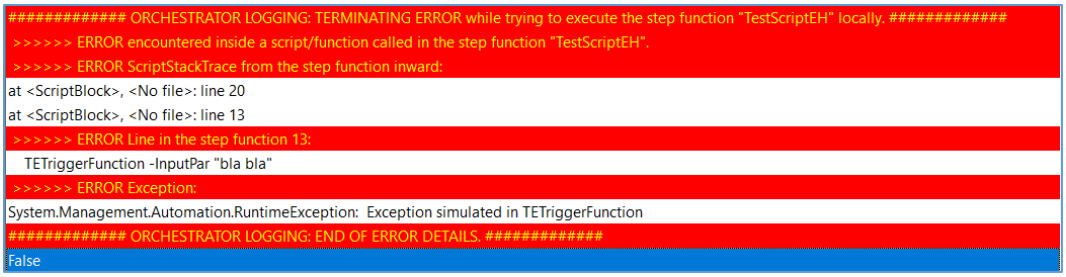
1. The error occurred inside a lower level (helper) function that is called in the step function, with the lower level function potentially being several levels down.

This case is identified by the first error detail line ">>>>>> ERROR encountered inside a script/function called in the step function […]". It can be seen in both pictures below.

The next message ">>>>>> ERROR ScriptStackTrace from the step function inward:" provide details about the depth of the ScriptStackTrace between the step function level (last line) and the called function that triggered the error (first line).

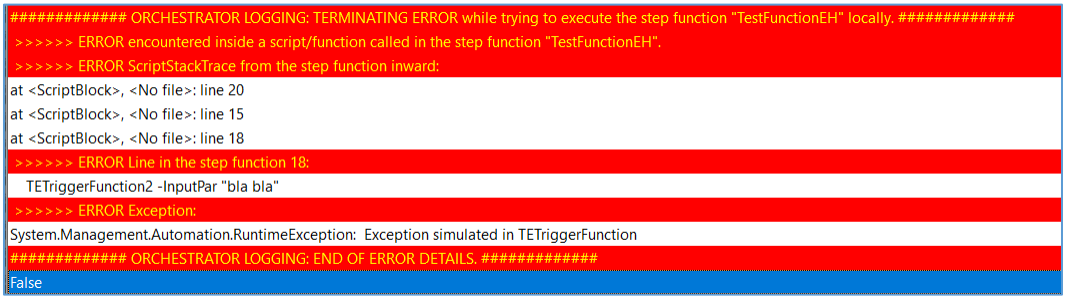
In the first picture there are 2 levels in the ScriptStackTrace:

- The last line "at <ScriptBlock>, <No file>: line 13" refers to the line inside the step function that triggered the error. This line can be actually inspected in the next message ">>>>>> ERROR Line in the step function:" and the line is " TETriggerFunction -InputPar "bla bla"".
- The first line "at <ScriptBlock>, <No file>: line 20" refers to the line inside the innermost called (helper) function that triggered the error. In this case there are only 2 levels, so the line number is from the (helper) function called in the step function, TETriggerFunction.



In the second picture there are 3 levels in the ScriptStackTrace:

- This means in the step function there is a called function and in that function there is another called function that triggered the error.
- The last line "at <ScriptBlock>, <No file>: line 18" refers to the line inside the step function that triggered the error. This line can be actually inspected in the next message ">>>>>> ERROR Line in the step function:" and the line is " TETriggerFunction2 -InputPar "bla bla"".
- The second line "at <ScriptBlock>, <No file>: line 15" is from the function called in the step function, "TETriggerFunction2", as deducted from the previous inspected line.
- The first line "at <ScriptBlock>, <No file>: line 20" refers to the line inside the innermost called (helper) function that triggered the error. In this case there are 3 levels, so the line number is from the innermost function, called inside the function "TETriggerFunction2".
- If the depth of the ScriptStackTrace is bigger than 3, the troubleshooting must go from called function to called function by visual code inspection, but with the help of the line numbers from each level in the ScriptStackTrace.



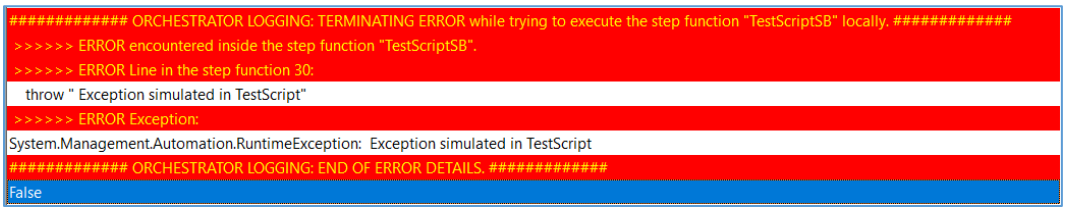
The following message is common in both cases above and refers to the error details from the innermost called function. The ">>>>>> ERROR Exception:" is the overall error description summary that usually appears in the console with red.

2. <u>The error occurred inside the step function</u>.

This case is identified by the first error detail line ">>>>>> ERROR encountered inside the step function "TestScriptSB"". It can be seen in the example below.

Since the error happened directly in the step function level, there is no ScriptStackTrace related message.

The remaining details are the same as in the cases above, regarding the Exception and the line in the step function that triggered the error.



At the end of all examples above we can see a “False” value added in the output. This is done by SPOT to mark the step execution as failed, since the step function itself cannot do that anymore, being terminated by the error.

4.6.2 Publishing step output as PV

It may be helpful sometimes to be able to export as a Published Variable the entire output from a step function. This content is already available inside the text log files from the “__SPOT_Artefacts” folder right after the execution has finished, as explained above, but in case this output is needed during the execution of the subsequent SPOT steps, exporting it to a Published Variable is not only possible but also easy. The step function output from every step type is found inside a SPOT internal variable called “_spot_Output”. This variable can be exported to a Published Variable, like seen in the below picture.

```
VariablesToPublish:
- _spot_Output=PsExecScriptErrorLine52
```

This includes any potential error details, as explained above.

Chapter 5 - Troubleshooting

This chapter is more of a placeholder as there is not much information available about troubleshooting due to the limited testing environments available at the moment for SPOT. In the next versions of the manual, this chapter will probably be extended.

- Local admin credentials not usable in remote PSSessions (it may be caused by a security feature enabled by default)

https://learn.microsoft.com/en-us/troubleshoot/windows-server/windows-security/disable-user-account-control

REG ADD HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 1

- WARNING: setting a step disabled in the yaml runbook config makes SPOT not to check it during runbook loading, so it may contain syntax errors even if the runbook is loaded successfully.
- When executing a runbook remotely some remote steps from the remote runbook may require for their credentials to have the domain part not empty and not ".", even if the same steps executed from a local runbook may accept that. This depends on the configuration at the OS level and is not imposed in any way from the SPOT tool. Overall, it is recommended to avoid using the "." domain in credentials used in combination with remote runbooks.
- When executing the SSH related SPOT built-in step functions remotely, the temporary SshNet dll files cannot be cleaned up as they are locked by the same PowerShell process that tries to delete them. This is a known issue and it manifests only in remote executions, which should be rare.
- Some SPOT internal functions do not provide normal output and only log directly to the default SPOT log file, C:\Windows\Temp\Orchestration.log. This is important to remember when troubleshooting issues that occur during remote step or runbook executions and to check the SPOT log file from the remote computers.